UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería

Curso: Construcción de Software

ReservaUPLA: Sistema de Gestión de Reservas de Ambientes Universitarios

Informe técnico del caso práctico (semanas 01 a 09)

Estudiante: _____

Docente: _____

Fecha: 10 de noviembre de 2026

Huancayo, Perú

Resumen 5
Cifras del proyecto 5
Dónde está cada criterio de evaluación del caso 5
1. Análisis del problema, actores y alcance (semana 01) 6
1.1 Problema 6
1.2 Actores 7
1.3 Alcance 7
1.4 Requerimientos funcionales y su evidencia 7
1.5 Reglas de negocio 8
1.6 Decisiones de análisis 9
2. Ciclo de vida del software (semana 02) 10
2.1 Modelo elegido: incremental e iterativo 10
2.2 Por qué encaja con este caso 10
2.3 Por qué no los otros modelos 10
2.4 Incrementos y su prueba de cierre 11
2.5 Riesgos y cómo se controlaron 11
3. Estándares de construcción (semana 03) 12
3.1 Nombres 12
3.2 Carpetas y dependencias entre capas 12
3.3 Commits y ramas 13
3.4 Calidad y cómo se verifica 14
3.5 Seguridad y secretos 14
4. Estructura inicial: backend, frontend y base de datos (semana 04) 15
4.1 Arquitectura por capas 15
4.2 Base de datos por terminal 17
4.3 Modelo de datos 19
4.4 Integridad en la base de datos 21
4.5 Estructura del frontend 23
5. Formularios de entrada de datos (semana 05) 24
5.1 Campos de cada formulario 24
5.2 Nueva reserva (el formulario principal) 24
5.3 Registrar ambiente y registrar usuario 27
5.4 Decisiones de usabilidad en los formularios 28
6. Validaciones en frontend y backend (semana 06) 30

6.1 Tres capas, un mismo mensaje 30
6.2 Dónde se aplica cada regla 32
6.3 Evidencia de las validaciones 36
6.4 Pruebas que rompen cada regla a propósito 38
7. Reutilización de componentes (semana 07) 39
7.1 Los nueve componentes mínimos 39
7.2 Más allá de los nueve 41
7.3 Cómo se evita que la duplicación vuelva 41
8. Refinamiento de código (semana 08) 41
8.1 Refactorizaciones 41
8.2 Defectos hallados al probar con datos reales 42
9. Módulos de salida de datos y dashboard (semana 09) 44
9.1 Listado de reservas (RF-05) 44
9.2 Detalle y cambio de estado (RF-06 y RF-07) 45
9.3 Dashboard (RF-08) 49
9.4 Más allá del mínimo 52
10. Calidad: pruebas, accesibilidad, usabilidad y mantenibilidad 55
10.1 Pruebas automáticas 55
10.2 Accesibilidad medida 55
10.3 Usabilidad: principios aplicados 56
10.4 Modo claro y oscuro 56
10.5 Mantenibilidad y escalabilidad: qué se puede afirmar 60
11. Reflexión final y conclusiones 61
11.1 La pregunta del caso 61
11.2 Respuesta 61
11.3 Qué mejoró en calidad, mantenibilidad y evolución 62
11.4 Limitaciones y trabajo pendiente 62
Anexo A. Cómo ejecutar el sistema 63
A.1 Requisitos 63
A.2 Base de datos (por terminal, desde la raíz del repositorio) 63
A.3 Secretos 63
A.4 Backend y frontend 63
A.5 Cuentas de prueba 64
A.6 Pruebas 64

A.7 Documentos de evidencia en el repositorio 64
Anexo B. Índice de figuras 65

Si el índice aparece vacío o desactualizado, haz clic derecho sobre él y elige «Actualizar campo» → «Actualizar toda la tabla». Word también puede preguntarlo al abrir el archivo.

Resumen

ReservaUPLA es una aplicación web para registrar, validar, consultar y visualizar las reservas de laboratorios, auditorios y salas de la Universidad Peruana Los Andes. Reemplaza los mensajes, las llamadas y los documentos físicos con un flujo único: el solicitante pide un ambiente, el sistema comprueba la disponibilidad, el administrador aprueba o rechaza y cada cambio queda registrado con quién lo hizo y cuándo.

Construcción. Backend en ASP.NET Core (.NET 8) con Clean Architecture, base de datos PostgreSQL creada por terminal con scripts SQL (sin migraciones) y frontend en React con Vite. Se desarrolló de forma incremental durante las nueve semanas: análisis, ciclo de vida, estándares, estructura, formularios, validaciones, reutilización, refinamiento y módulos de salida de datos.

Cifras del proyecto

Dato	Valor	Cómo se obtuvo
Requerimientos funcionales del caso	8 de 8 (RF-01 a RF-08)	Sección 1.4, con el archivo y la línea de cada uno
Reglas de negocio del caso	6 de 6 (RN-01 a RN-06), más 2 agregadas (RN-07 y mantenimiento)	Sección 6, aplicadas en frontend, backend y base de datos
Componentes reutilizables mínimos	9 de 9	Sección 7, con el número de archivos que usa cada uno
Indicadores mínimos del dashboard	5 de 5, más 2 de gestión	Sección 9
Commits en el repositorio	114	git rev-list (rama actual: feature/diseno-final)
Archivos de código	173 de frontend (sin pruebas) y 71 de backend (sin pruebas)	Contados al generar este informe
Pruebas automáticas	1375 de frontend, 176 de backend medidas el 2026-11-10 y 60 de la API (última ejecución completa el 2026-11-07)	Sección 10.1, con comando, fecha y límites de cada ejecución

Dónde está cada criterio de evaluación del caso

La tabla sigue los criterios del enunciado (20 puntos) y dice en qué sección está la evidencia de cada uno.

Criterio del caso	Puntaje	Sección	Evidencia principal
Análisis del problema, actores y alcance	2	1	Problema, actores, alcance, RF y RN con su evidencia
Justificación del ciclo de vida	2	2	Modelo incremental e iterativo, comparación con otros tres modelos
Aplicación de estándares de construcción	2	3	Nombres, carpetas, commits y ramas, y su verificación
Implementación de formularios de entrada	3	5	Tres formularios (usuario, ambiente, reserva) con capturas
Validaciones frontend y backend	3	6	Cada regla en tres capas, con la prueba que la rompe a propósito
Reutilización de componentes	2	7	Los nueve componentes, su uso y la prueba que impide duplicar
Refinamiento de código	2	8	Refactorizaciones, defectos hallados y corregidos
Módulos de salida y dashboard	3	9	Listado, detalle, dashboard por rol
Presentación y sustentación	1	Anexo A y 11	Cómo ejecutarlo y la reflexión final

Los entregables del punto 11 del enunciado están así: repositorio (sección 3 y Anexo A), backend funcional (secciones 4 y 6), frontend funcional (secciones 5, 7 y 9), base de datos implementada (sección 4), evidencia de validaciones (sección 6), capturas del dashboard (sección 9), este informe y la sustentación.

1. Análisis del problema, actores y alcance (semana 01)

1.1 Problema

La Universidad Peruana Los Andes administra laboratorios de cómputo, auditorios, salas de reuniones y laboratorios especializados. Hoy las reservas se piden por mensajes, llamadas o documentos físicos. El caso identifica cuatro consecuencias, y cada una se atiende con una parte concreta del sistema.

Consecuencia reportada	Causa	Cómo la atiende ReservaUPLA
Duplicidad de reservas	Nadie comprueba que el ambiente esté libre a esa hora	RF-04 y RN-01: se valida la disponibilidad y la base de datos impide dos reservas aprobadas que se crucen

Consecuencia reportada	Causa	Cómo la atiende ReservaUPLA
Pérdida de solicitudes	La solicitud vive en un mensaje o un papel	RF-03 y RF-05: toda solicitud queda registrada y se puede listar y filtrar
Falta de trazabilidad	No se sabe quién pidió, quién aprobó ni cuándo	RF-06 y RF-07: detalle con su historial de estados, y «Actividad reciente» en el dashboard
Dificultad para conocer la disponibilidad real	No hay una vista única	RF-05 y RF-08: listado con filtros, dashboard y vista diaria de disponibilidad

1.2 Actores

Actor	Responsabilidad (según el caso)	Qué necesita del sistema
Solicitante	Registra solicitudes de reserva	Pedir un ambiente, saber si está libre, ver el estado de sus solicitudes y cancelarlas
Administrador	Aprueba o rechaza reservas	Ver las solicitudes pendientes, decidir con la seguridad de que no habrá cruces y ver el panorama general
Responsable de ambiente	Consulta el uso del ambiente asignado	Ver las reservas de su ambiente, sin poder aprobarlas

El caso habla de «ambiente asignado», pero el modelo mínimo no dice a quién está asignado un ambiente. Por eso el modelo se amplía con `ResponsableId` en Ambiente (sección 4.3).

1.3 Alcance

Incluye: registro de usuarios, ambientes y reservas; validación de disponibilidad; listado y detalle de reservas; cambio de estado; dashboard con los cinco indicadores; roles y control de permisos.

No incluye (queda fuera por no ser parte del reto): notificaciones por correo, reservas recurrentes, calendario semanal o mensual, exportación a PDF o Excel, recuperación de contraseña y varias sedes. La vista diaria de disponibilidad apoya RF-04 y RF-08, pero no es el calendario excluido: es una línea de tiempo acotada al día elegido.

1.4 Requerimientos funcionales y su evidencia

Cada requerimiento del caso con el archivo y la línea donde se implementa (comprobados al generar este informe).

Código	Requerimiento	Backend	Frontend
RF-01	Registrar usuarios solicitantes	backend/ReservaUPLA.API/ Controllers/ UsuariosController.cs:38	frontend/src/pages/ UsuarioFormPage.jsx:12
RF-02	Registrar ambientes universitarios	backend/ReservaUPLA.API/ Controllers/ AmbientesController.cs:25	frontend/src/pages/ AmbienteFormPage.jsx:10
RF-03	Registrar solicitudes de reserva	backend/ReservaUPLA.API/ Controllers/ ReservasController.cs:78	frontend/src/pages/ NuevaReservaPage.jsx:34
RF-04	Validar disponibilidad antes de registrar	backend/ReservaUPLA.API/ Controllers/ ReservasController.cs:64	frontend/src/hooks/ useDisponibilidad.js:26
RF-05	Listar reservas registradas	backend/ReservaUPLA.API/ Controllers/ ReservasController.cs:32	frontend/src/pages/ ReservasPage.jsx:31
RF-06	Consultar detalle de una reserva	backend/ReservaUPLA.API/ Controllers/ ReservasController.cs:57	frontend/src/pages/ DetalleReservaPage.jsx:16
RF-07	Cambiar el estado: Pendiente, Aprobada, Rechazada o Cancelada	backend/ReservaUPLA.API/ Controllers/ ReservasController.cs:91	frontend/src/pages/ DetalleReservaPage.jsx:16
RF-08	Mostrar indicadores básicos en un dashboard	backend/ReservaUPLA.API/ Controllers/ PanelesController.cs:24	frontend/src/pages/ InicioAdministrador.jsx:54

1.5 Reglas de negocio

Una regla importante nunca se valida en un solo sitio: el frontend ayuda al usuario, el backend decide y la base de datos es la última barrera. La sección 6 detalla cada regla con su prueba. El caso pide RN-01 a RN-06; se agregaron dos reglas razonadas y se dejaron registradas en el análisis.

Regla	Enunciado
RN-01	No más de una reserva aprobada para el mismo ambiente, fecha y horario
RN-02	Toda reserva registra fecha, hora de inicio, hora de fin, ambiente, solicitante y motivo
RN-03	La hora de fin es mayor que la de inicio

Regla	Enunciado
RN-04	Solo entre las 07:00 y las 22:00
RN-05	Solo el administrador aprueba o rechaza
RN-06	Una reserva rechazada no se modifica (también es final la cancelada, decisión propia)
RN-07 (agregada)	No se reserva para una fecha anterior a hoy (hoy es la fecha de Lima); solo se controla al registrar, nunca al cambiar de estado
Mantenimiento (agregada)	Un ambiente en mantenimiento no admite reservas nuevas

Transiciones de estado permitidas:

Desde	Hacia	Quién
Pendiente	Aprobada	Administrador, solo si no hay cruce con otra aprobada (RN-01)
Pendiente	Rechazada	Administrador
Pendiente o Aprobada	Cancelada	El solicitante dueño de la reserva o el administrador
Rechazada, Cancelada	(ninguna)	Estados finales (RN-06)

1.6 Decisiones de análisis

- **Inicio de sesión mínimo con rol.** El caso no lo pide, pero sin identificar al usuario no se puede cumplir RN-05 en el backend.
- **La disponibilidad se valida solo contra reservas aprobadas**, que es lo que dice RN-01. Las pendientes pueden coexistir y el administrador aprueba una y rechaza la otra.
- **«Ambiente más utilizado»** es el que tiene más reservas aprobadas; si hay empate se muestran todos los empatados.
- **RN-07 y mantenimiento** se agregaron por sentido común del dominio. Se documentó que su cumplimiento está en las tres capas y que RN-07 no se aplica al cambiar de estado para no bloquear reservas antiguas.

La restricción de RN-01 en la base de datos está en `database/03_crear_tablas.sql:57` y se explica en la sección 6.

2. Ciclo de vida del software (semana 02)

2.1 Modelo elegido: incremental e iterativo

El sistema se construyó en **incrementos**: cada uno entrega algo que funciona y se prueba, y después de cada incremento se revisa y se corrige antes de continuar. Las semanas 04 a 09 del caso ya están planteadas así: estructura, formularios, validaciones, reutilización, refinamiento y salida de datos.

2.2 Por qué encaja con este caso

Característica del caso	Qué implica	Por qué responde bien el modelo incremental
El enunciado pide desarrollar «progresivamente» durante nueve semanas	Hay entregas parciales con evidencia cada semana	Cada incremento es una entrega comprobable
Los RF y las RN están definidos desde el inicio	No hace falta descubrirlos sobre la marcha	Se pueden planificar los incrementos con anticipación
Hay dependencias claras: los formularios necesitan componentes y el dashboard necesita datos de reservas	El orden importa	Cada incremento se apoya en el anterior
La regla más delicada (RN-01) afecta a todo el sistema	Un error aquí se descubriría tarde	Se aborda pronto y se vuelve a probar en cada incremento
El proyecto lo desarrolla una sola persona	No hay equipo ni roles de gestión	Un modelo ligero evita ceremonias que no aportan

2.3 Por qué no los otros modelos

Modelo	Motivo para descartarlo
Cascada	Entrega todo al final: si un error de análisis aparece en el dashboard, obliga a rehacer lo anterior. Además contradice la evidencia semanal que pide el caso
Espiral	Está pensado para proyectos con riesgos grandes y costosos; aquí el alcance es pequeño y conocido
Scrum	Requiere equipo, roles (Product Owner, Scrum Master) y reuniones; una sola persona no puede aplicarlo con sentido

2.4 Incrementos y su prueba de cierre

Incremento	Semana	Resultado que funciona	Prueba al cerrar
0. Análisis y estándares	01 a 03	Documentos acordados	Revisión del alcance
1. Estructura inicial	04	Backend, frontend y base de datos que arrancan y se comunican	La base se crea desde cero con los scripts y la API responde
2. Entrada de datos	05	Formularios para usuarios, ambientes y reservas	Se puede registrar cada entidad
3. Validación	06	RN-01 a RN-06 en frontend, backend y base de datos	Cada regla tiene una prueba que la rompe a propósito
4. Reutilización	07	Los nueve componentes compartidos usados en todas las pantallas	Ninguna pantalla repite código de campos o mensajes
5. Refinamiento	08	Servicios, controladores y validaciones reorganizados	Las pruebas siguen en verde
6. Salida de datos	09	Listado, detalle y dashboard	Los números del dashboard coinciden con la base

2.5 Riesgos y cómo se controlaron

Riesgo	Control aplicado
Dos solicitudes simultáneas se aprueban para el mismo horario	Restricción de exclusión en la base de datos, además de la validación en el backend; prueba de concurrencia con la base real
Reglas distintas en frontend y backend	Mismos mensajes con una sola fuente en cada lado y una prueba que falla si difieren
Que el dashboard muestre cifras que no coinciden con los datos	Pruebas del servicio de consulta y revisión con los datos reales de demostración

Una iteración posterior, ya terminado el incremento 6, agregó una segunda pasada de diseño y accesibilidad (rama `feature/experiencia`, `feature/pulido-visual`, `feature/taste` y `feature/diseno-final`), con sus propias mediciones. Es el modelo incremental aplicado de nuevo: se mejora sobre lo que ya funciona y cada pasada cierra con pruebas.

3. Estándares de construcción (semana 03)

El manual completo está en `docs/03-estandares.md`. Son reglas breves y comprobables; si una regla no se puede verificar, no está en el manual. Aquí se resumen y se muestra cómo se verifica cada una.

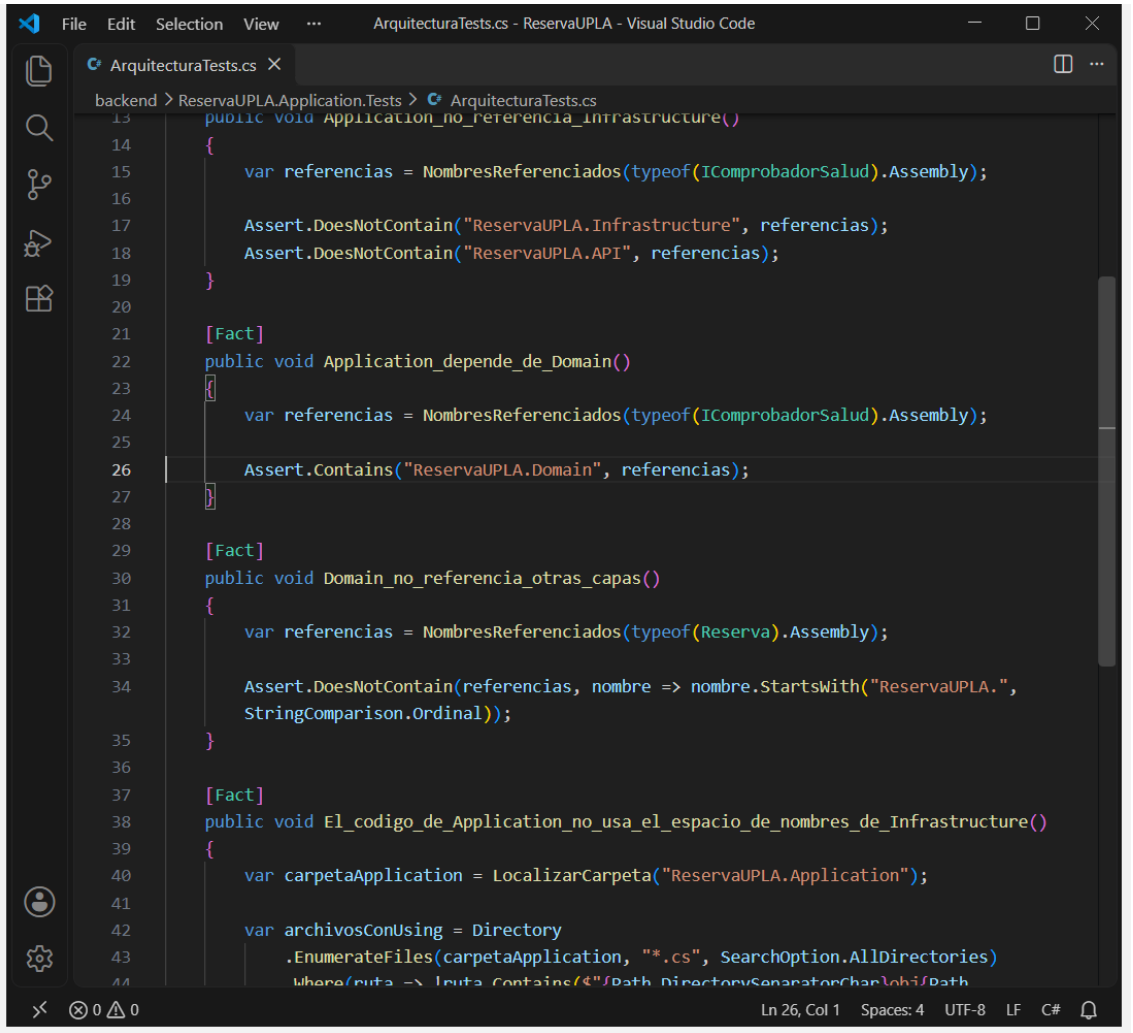
3.1 Nombres

Elemento	Regla	Ejemplo real
Dominio	En español	`Reserva`, `Ambiente`, `HoraInicio`
Términos técnicos	En inglés cuando son convención	`ReservaController`, `ReservaRepository`, `ReservaService`
C#	PascalCase para clases y métodos; camelCase para variables; prefijo `I` en interfaces	`IReservaRepository`, `TransicionarAsync`
React	PascalCase, un componente por archivo; hooks `useXxx`	`TextInput.jsx`, `useReservaForm.js`
SQL	snake_case en minúsculas, tablas en plural	`reservas`, `hora_inicio`

3.2 Carpetas y dependencias entre capas

Esta prueba lee las referencias del proyecto Application y falla si aparece Infrastructure. Así la regla de dependencias deja de ser una promesa y pasa a ser algo que el compilador de pruebas vigila en cada cambio.

Figura 1. Prueba de arquitectura: Application no puede referenciar Infrastructure (backend/ReservaUPLA.Application.Tests/ArquitecturaTests.cs, líneas 12–21)



- backend/ (Domain, Application, Infrastructure, API y proyectos de pruebas), `frontend/` (`components/ui`, `pages`, `hooks`, `services`, `utils`, `constants`), `database/` (scripts SQL numerados) y `docs/`.
- **Application depende de Domain; Infrastructure depende de Application y Domain; API depende de Application e Infrastructure. Application nunca referencia Infrastructure.** Lo comprueba una prueba de arquitectura (`ArquitecturaTests.cs`).

3.3 Commits y ramas

Formato `tipo: descripción en presente y en español` (feat, fix, refactor, test, docs, chore), un cambio por commit y sin ámbito entre paréntesis. El repositorio tiene **114 commits**: `feat` 37, `docs` 30, `fix` 16, `merge` 9, `test` 8, `chore` 7, `refactor` 5, `style` 2. Las ramas son `feature/<tema>`: `feature/diseno-final`, `feature/experiencia`, `feature/pulido-visual`, `feature/reutilizacion`, `feature/salida-de-datos`, `feature/taste`, `feature/usabilidad`, `main`.

3.4 Calidad y cómo se verifica

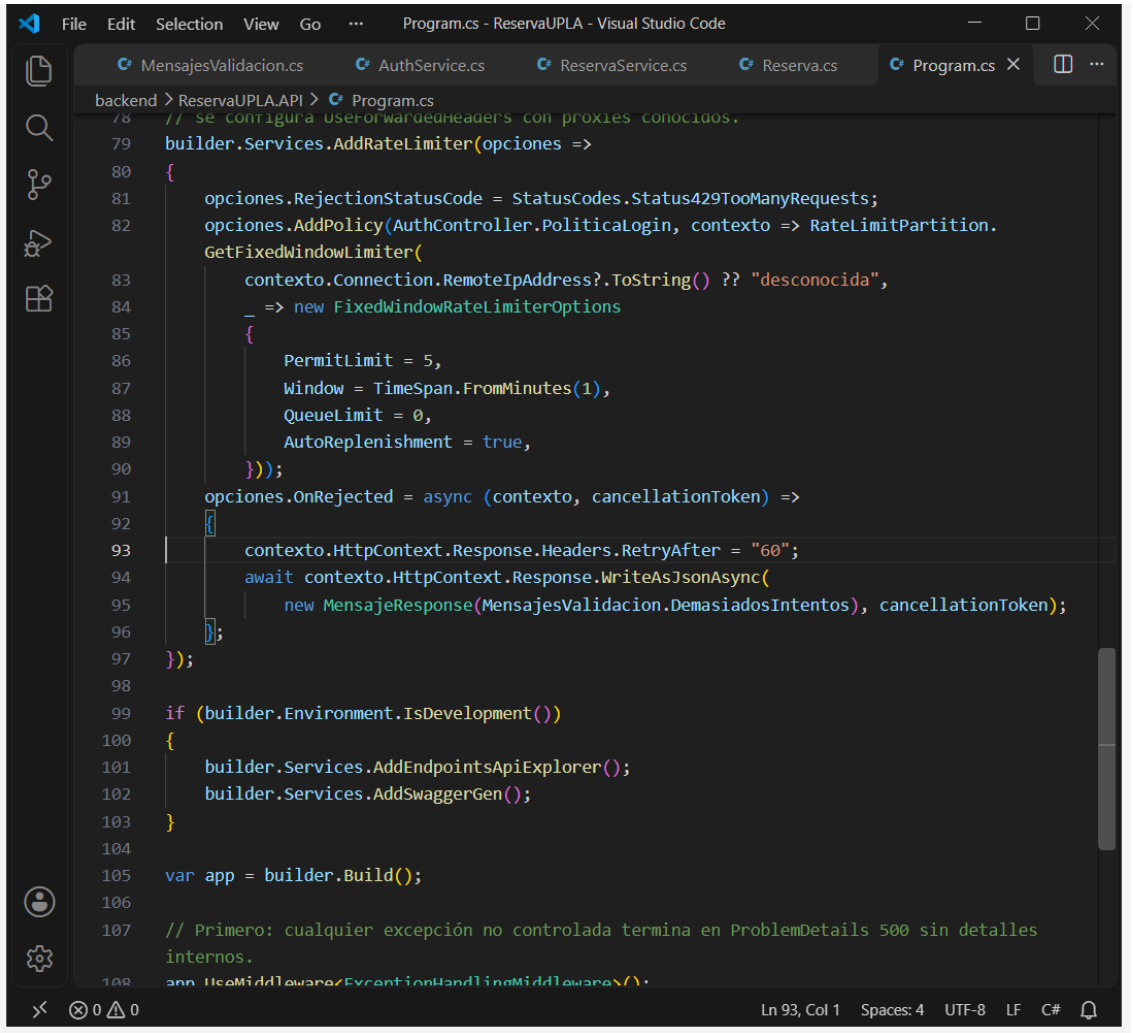
Verificación	Herramienta	Criterio	Resultado
Estilo del frontend	ESLint y Prettier	`--max-warnings 0` y sin cambios pendientes	0 advertencias; formato sin diferencias (2026-11-10)
Pruebas	xUnit y Vitest	Todas aprobadas antes de unir a `main`	Sección 10.1
Una sola cosa por archivo	Revisión y límite de 300 líneas	Ningún archivo de código (JavaScript, JSX o C#) supera 300 líneas	El más largo es `components/Disponibilidad/BloquesDisponibilidad.jsx` con 287 líneas en frontend y `ReservaService.cs` con 289 en backend. Las hojas de estilo de pantalla sí pasan de 300 (la mayor tiene 486): es una excepción conocida
Sin HTML suelto	Prueba `arquitectura.test.js`	Ninguna pantalla escribe `<input>`, `<select>`, `<textarea>` o `<button>` fuera de `components/ui`	Aprobada
Mensajes sin repetir	Prueba `messages.test.js`	Cada texto de interfaz está en `messages.js` y coincide con el backend	Aprobada
Colores sin repetir	Prueba `identidad.test.js`	Ningún color, radio ni sombra fuera de `tokens.css`	Aprobada

La regla de «una sola fuente» para los colores es la que permitió agregar el **modo oscuro** al final del proyecto sin tocar los componentes: ver `frontend/src/styles/tokens.css:123`.

3.5 Seguridad y secretos

Frena la fuerza bruta contra el login: el sexto intento en un minuto recibe un 429 con un mensaje que explica cuánto esperar.

Figura 2. Límite de intentos del inicio de sesión: 5 por minuto por dirección IP (backend/ReservaUPLA.API/Program.cs, líneas 79–92)



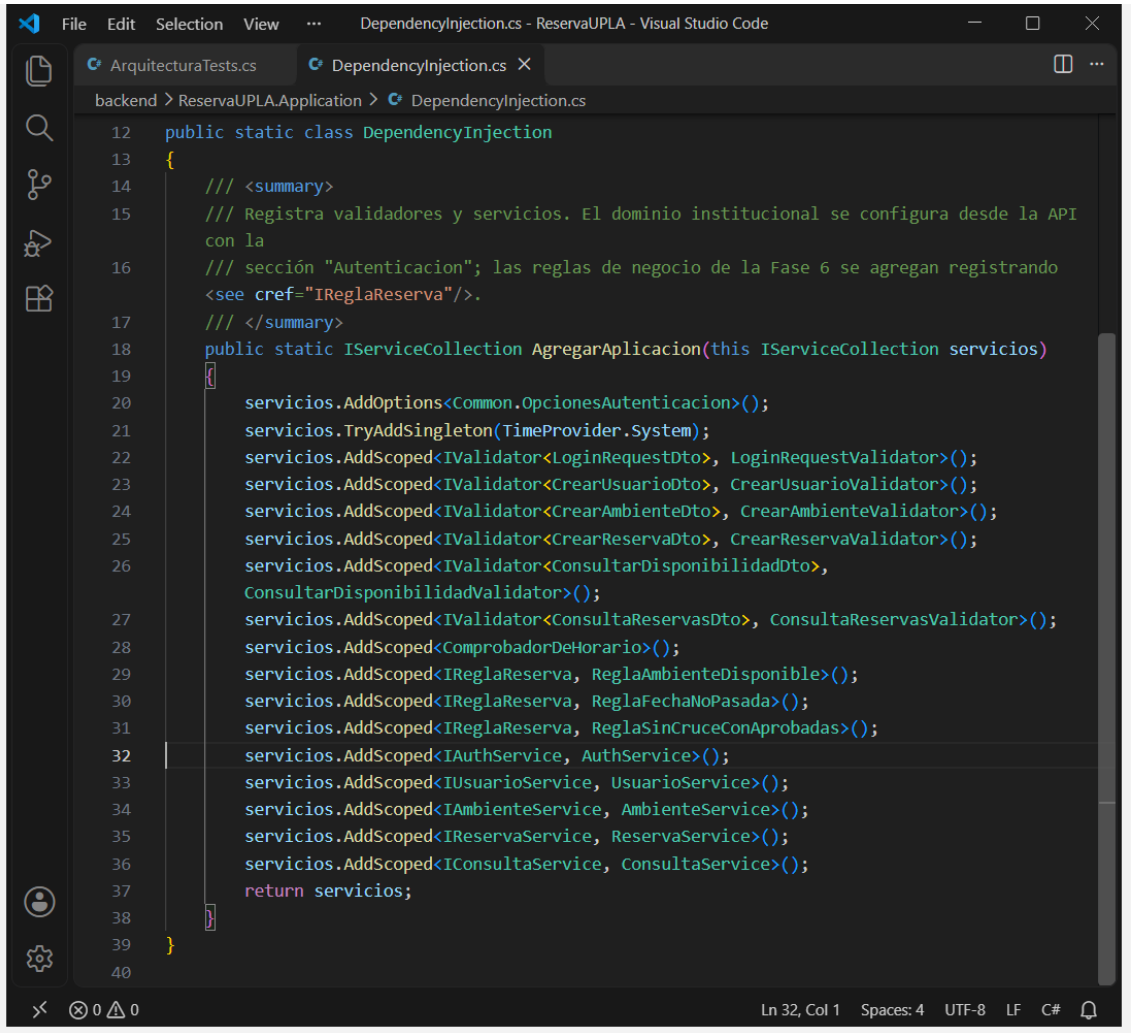
- Contraseñas siempre como hash BCrypt (la base de datos rechaza un valor que no tenga ese formato), nunca en texto plano ni en los registros.
- Cadena de conexión y clave del token JWT en `dotnet user-secrets` o variables de entorno; nunca en el repositorio. Los scripts SQL reciben las claves como variables de `psql`.
- Los mensajes de error no revelan información interna; el login responde lo mismo para un usuario inexistente, inactivo o con contraseña errónea y limita los intentos (5 por minuto).

4. Estructura inicial: backend, frontend y base de datos (semana 04)

4.1 Arquitectura por capas

Cada capa expone un único método de extensión que registra sus piezas; Program.cs solo llama a esos métodos y queda corto. Application registra interfaces (por ejemplo, IValidator) y no conoce a Infrastructure.

Figura 3. Registro de validadores y servicios de la capa Application (backend/ReservaUPLA.Application/DependencyInjection.cs, líneas 18–31)



El backend sigue Clean Architecture: el negocio no depende de la base de datos ni del framework web, y las dependencias apuntan siempre hacia adentro. Cada capa es un proyecto de C# propio, de modo que el compilador impide referencias indebidas.

Capa (proyecto)	Qué contiene	Depende de
`ReservaUPLA.Domain`	Entidades (`Reserva`, `Ambiente`, `Usuario`, `HistorialReserva`) y sus reglas: las transiciones de estado viven en la propia entidad	Nada
`ReservaUPLA.Application`	Servicios de caso de uso, validadores (FluentValidation), DTOs, interfaces de repositorio y reglas de negocio	Domain
`ReservaUPLA.Infrastructure`	Entity Framework Core con Npgsql, repositorios, BCrypt y generación del token JWT	Application y Domain

Capa (proyecto)	Qué contiene	Depende de
`ReservaUPLA.API`	Controladores delgados (traducen resultados a HTTP), autenticación JWT, límite de intentos, CORS y manejo de errores	Application e Infrastructure
Pruebas	`Application.Tests` (repositorios falsos), `Infrastructure.Tests` y `API.Tests` (base real)	—

La regla «Application nunca referencia Infrastructure» se aprecia en las referencias de proyecto (`backend/ReservaUPLA.Application/ReservaUPLA.Application.csproj:4`, que solo apunta a Domain) y se verifica con la prueba `ArquitecturaTests.cs`.

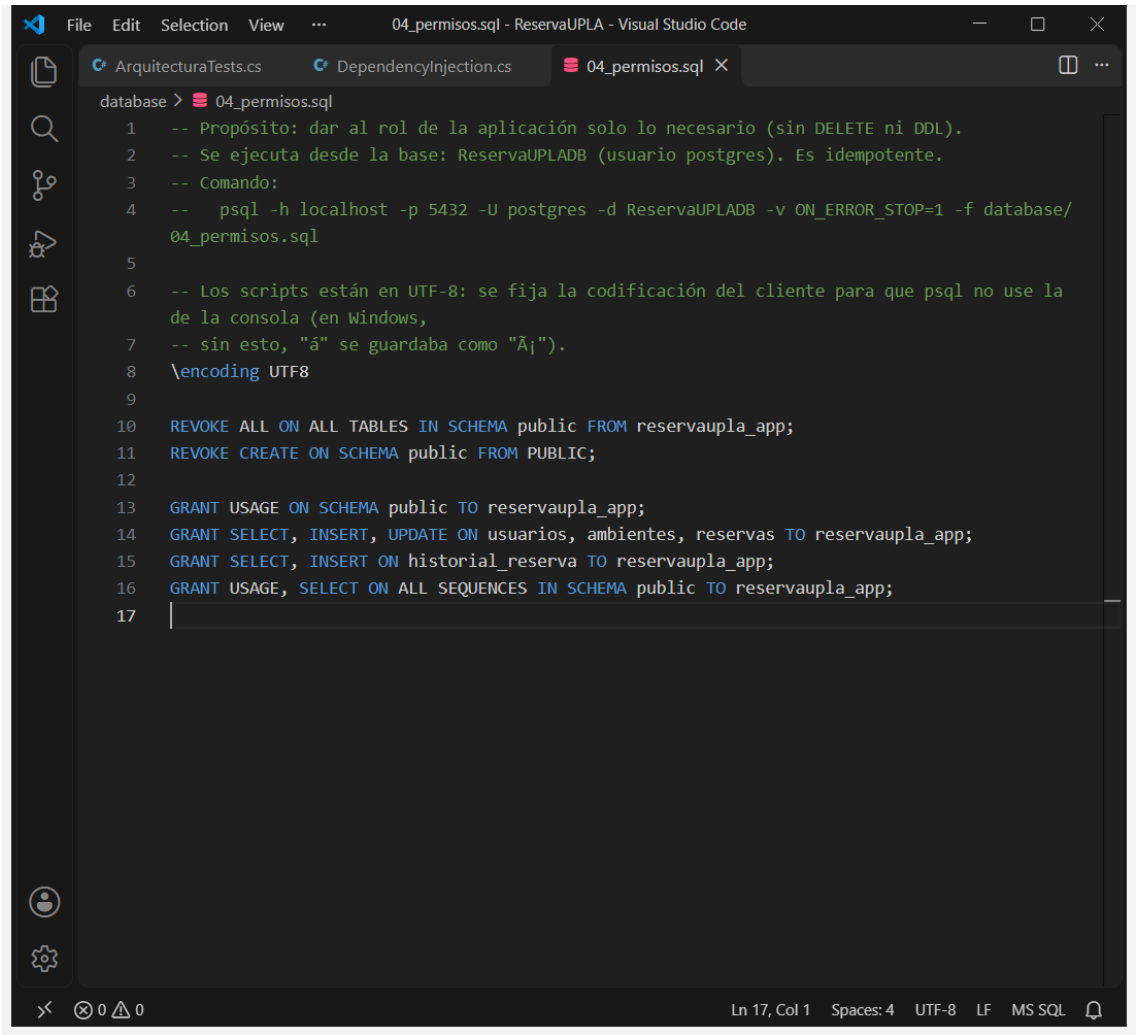
Las reglas que más importan viven en el dominio y no en el controlador. Por ejemplo, una reserva solo puede aprobarse si está pendiente, y el propio objeto lo exige:

`backend/ReservaUPLA.Domain/Entities/Reserva.cs:41–45`: `Aprobar`, `Rechazar` y `Cancelar` llaman a `ExigirPendiente` o `ExigirNoFinal` antes de cambiar el estado y registran el evento de historial en la misma operación.

4.2 Base de datos por terminal

La API no se conecta como superusuario: el rol reservaupla_app solo puede leer y escribir lo que el sistema necesita, y el historial es de solo agregar (no se actualiza ni se borra).

Figura 4. Permisos del rol de la aplicación (mínimo privilegio) (database/04_permisos.sql, líneas 12–17)



La base de datos, las tablas, los permisos y los datos de prueba se crean **por terminal con psql** y scripts SQL versionados en `database/`. No se usan migraciones ni se crean tablas desde el código: el esquema queda legible, revisable y reproducible desde cero.

Script	Qué hace
`00_eliminar_bd.sql`	Reinicio opcional: borra la base y el rol
`01_crear_bd.sql`	Crea `ReservaUPLADB` en UTF-8
`02_crear_rol.sql`	Crea el rol `reservaupla_app` con mínimo privilegio (la clave llega como variable de `psql`)
`03_crear_tablas.sql`	Tablas, restricciones, índices y disparadores
`04_permisos.sql`	Concede al rol solo lo necesario (leer, insertar y actualizar donde corresponde)

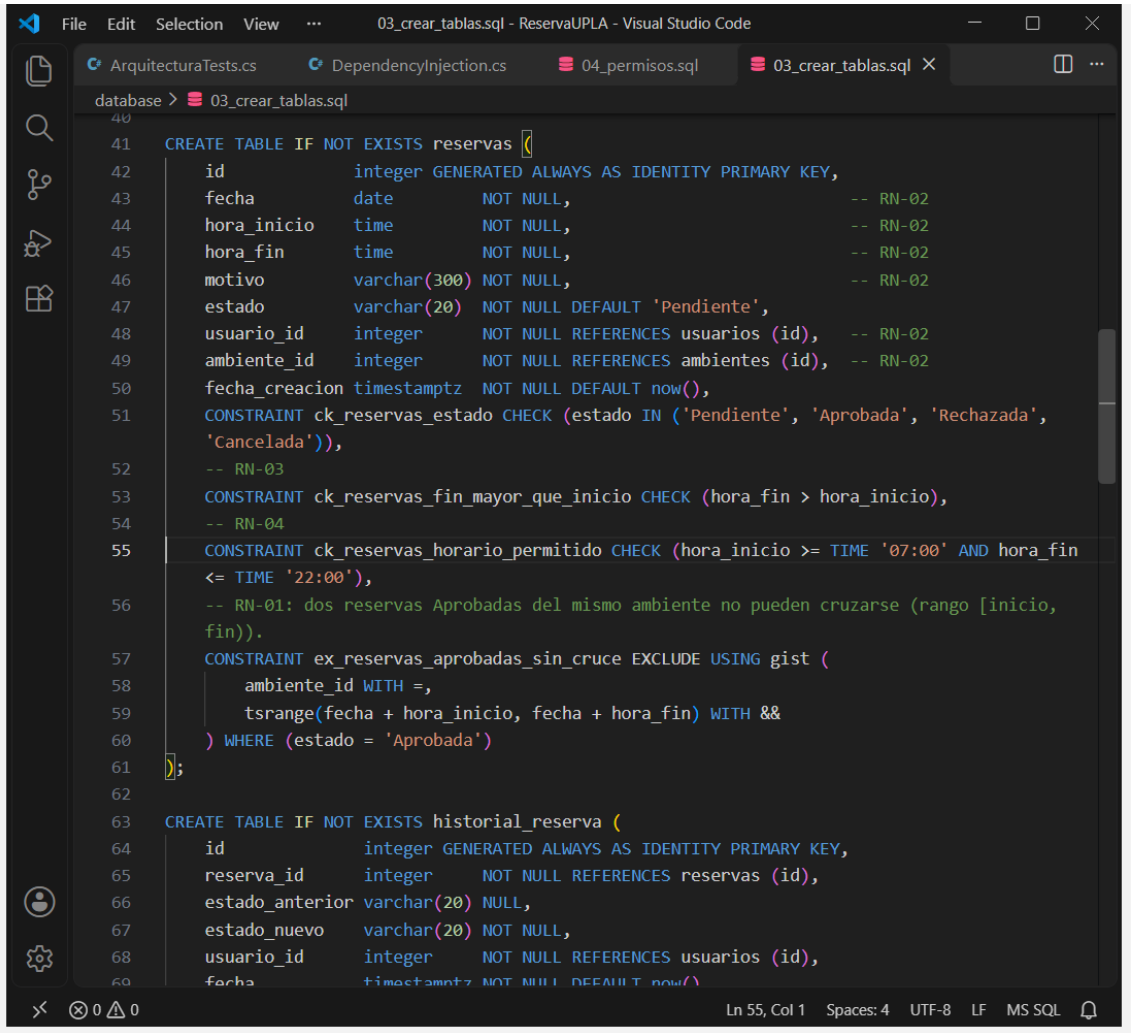
Script	Qué hace
`05_datos_prueba.sql`	Tres usuarios, tres ambientes y reservas de prueba con contraseñas en BCrypt
`06_verificacion.sql`	Comprueba el esquema y que las tildes estén bien codificadas
`07_pruebas_reglas.sql`	Intenta romper RN-01 a RN-07 y termina en ROLLBACK: si una regla no se rechaza, el script falla
`07_datos_demo.sql`	Opcional: reservas con fechas relativas a hoy para una demostración completa

La aplicación se conecta con el rol `reservaupla_app`, no con `postgres`. Los comandos exactos para crear la base están en el Anexo A.

4.3 Modelo de datos

La base de datos también hace cumplir las reglas: RN-02 con NOT NULL, RN-03 y RN-04 con CHECK, y RN-01 con una restricción de exclusión que impide que dos reservas aprobadas del mismo ambiente se crucen, incluso si dos personas aprueban a la vez.

Figura 5. Tabla reservas: restricciones NOT NULL, CHECK y de exclusión (database/03_crear_tablas.sql, líneas 41–61)



Se partió de las tres entidades del enunciado y se amplió lo justificado. Las diferencias con el modelo «sugerido» del caso son deliberadas y se explican a continuación.

Entidad	Del enunciado	Ampliación	Motivo
Usuario	Id, NombreCompleto, Correo, Rol, Activo	`contrasena_hash`	Iniciar sesión para que RN-05 se cumpla de verdad
Ambiente	Id, Nombre, Tipo, Capacidad, Ubicacion, Estado	`responsable_id` (opcional)	El responsable consulta «su» ambiente
Reserva	Id, Fecha, HoraInicio, HoraFin, Motivo, Estado, UsuarioId, AmbienteId	`fecha_creacion`	Ordenar por antigüedad las solicitudes pendientes

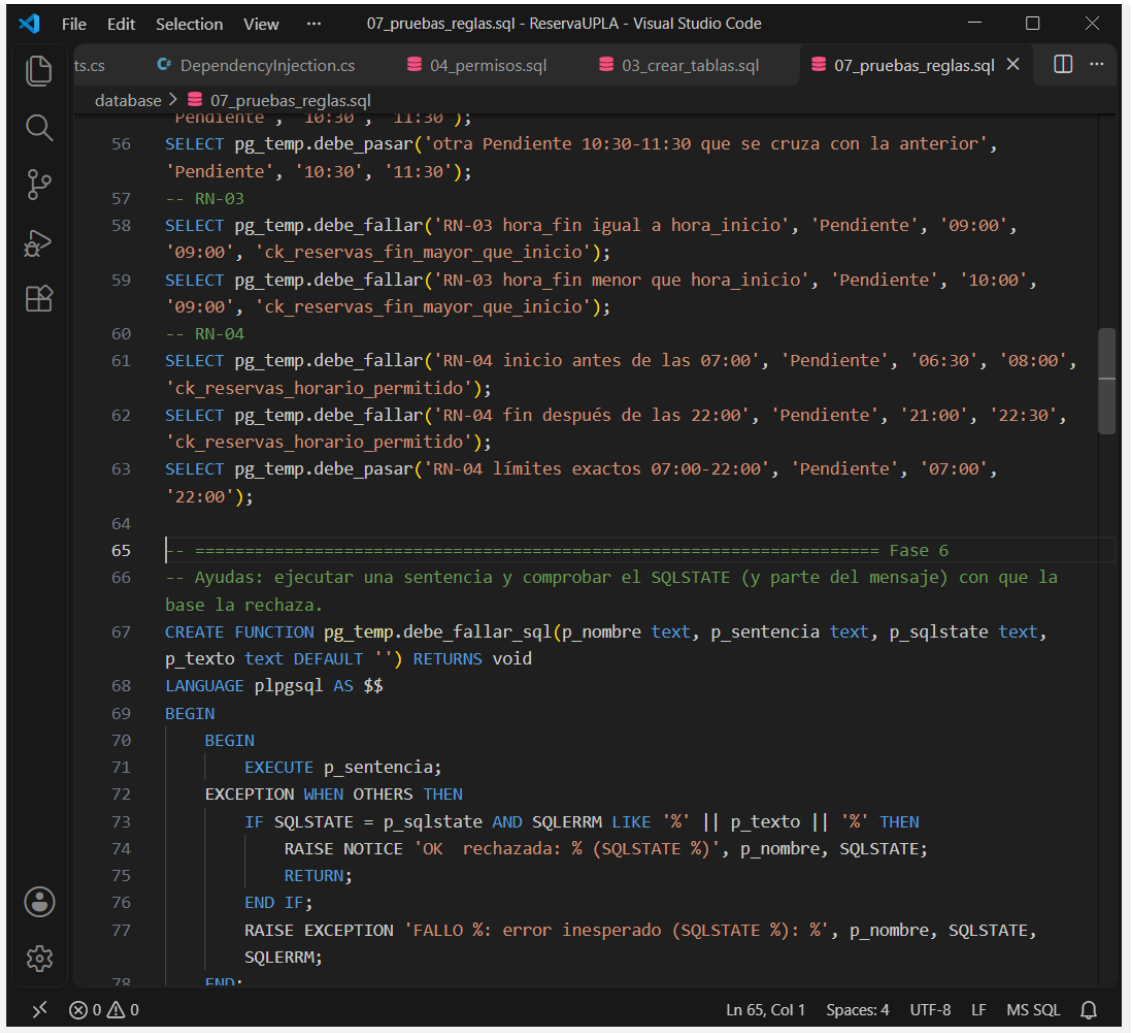
Entidad	Del enunciado	Ampliación	Motivo
HistorialReserva (nueva)	—	reserva, estado anterior y nuevo, quién y cuándo	Responde al problema de «falta de trazabilidad»

Decisión sobre los tipos. El enunciado sugiere `UUID` para los identificadores y longitudes como `VARCHAR(150)`. Se usaron identificadores enteros autoincrementales (`integer GENERATED ALWAYS AS IDENTITY`) y longitudes ajustadas a los límites reales de cada campo (por ejemplo, nombre de 120 y motivo de 300 caracteres). El caso dice que el tipo es «sugerido» y que el modelo se puede ampliar. La razón es que no hay ninguna necesidad de generar identificadores fuera de la base de datos y los enteros hacen las consultas y las claves foráneas más simples y rápidas. Si el sistema creciera a varias sedes con bases distintas, cambiar a UUID sería una decisión a tomar entonces.

4.4 Integridad en la base de datos

Este script intenta romper las reglas a propósito (una segunda reserva aprobada que se cruza) y comprueba que la base responde con el error esperado; termina en ROLLBACK, por lo que no deja datos.

Figura 6. Script que prueba las reglas directamente en PostgreSQL (database/07_pruebas_reglas.sql, líneas 51–56)



La base de datos también protege las reglas (defensa en profundidad): aunque alguien llame a la API sin pasar por el frontend o escriba directamente en las tablas, los datos inválidos se rechazan.

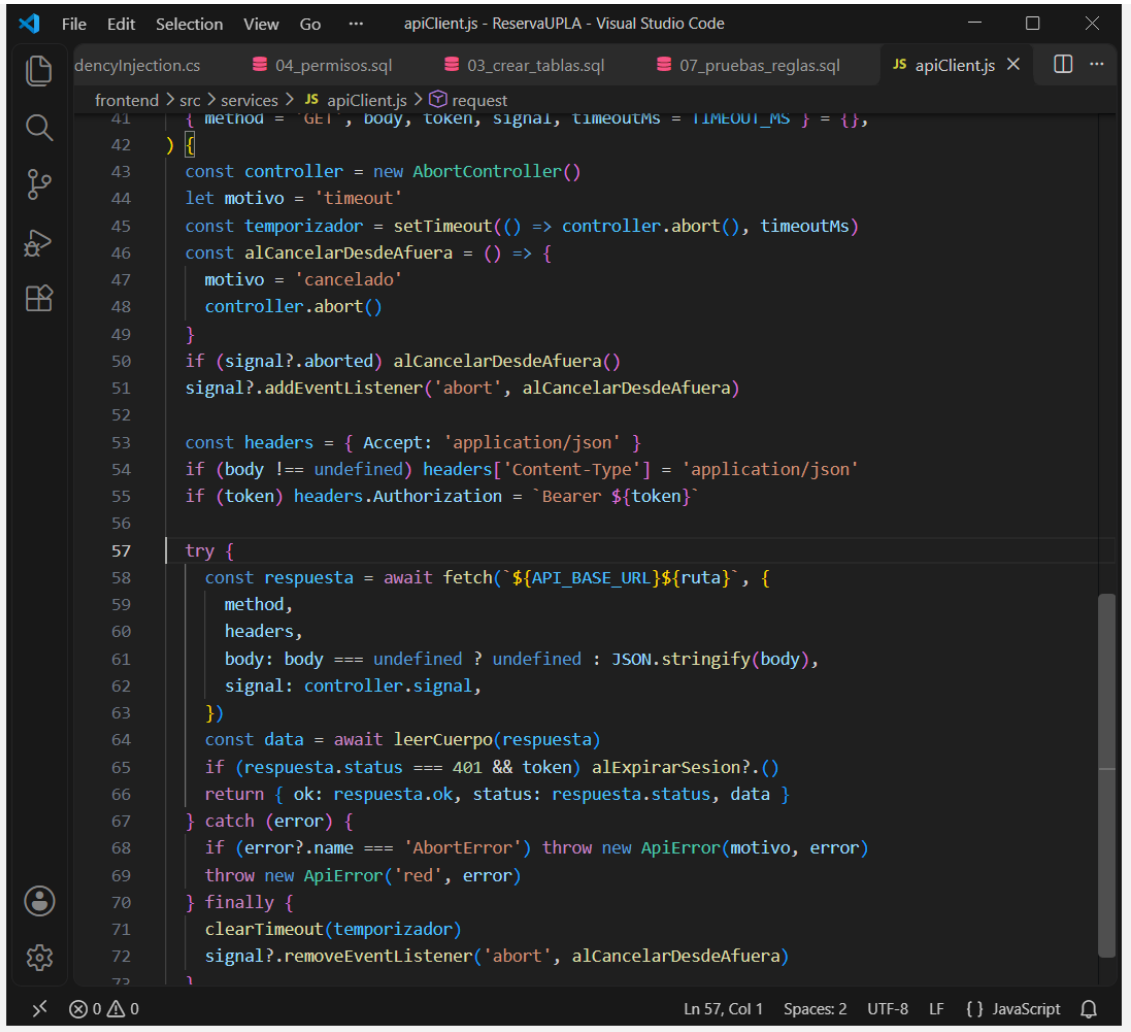
Protección	Dónde
Restricción de exclusión: dos reservas aprobadas del mismo ambiente no pueden cruzarse (RN-01)	database/03_crear_tablas.sql:57
Fin mayor que inicio (RN-03)	database/03_crear_tablas.sql:53
Horario permitido de 07:00 a 22:00 (RN-04)	database/03_crear_tablas.sql:55
Campos obligatorios (RN-02): `NOT NULL` en fecha, horas, motivo, usuario y ambiente	database/03_crear_tablas.sql:46

Protección	Dónde
Solo se guarda un hash BCrypt, nunca texto plano	database/03_crear_ tablas.sql:24
Transiciones de estado válidas (RN-06): Rechazada y Cancelada son finales	database/03_crear_ tablas.sql:81
No se reserva en un ambiente en mantenimiento ni en una fecha pasada (RN-07)	database/03_crear_ tablas.sql:104

4.5 Estructura del frontend

Todas las llamadas a la API pasan por esta función: añade el token, corta la petición a los 15 segundos y distingue entre «sin respuesta» (red o tiempo) y una respuesta del servidor, para mostrar el mensaje correcto.

Figura 7. Cliente HTTP del frontend: tiempo límite, cancelación y errores de red (frontend/src/services/apiClient.js, líneas 43–70)



El frontend usa React con Vite y se organiza por responsabilidad: `components/ui` (los componentes compartidos), `components/<Pantalla>` (piezas de una pantalla), `pages` (una por ruta), `hooks` (estado y envío de formularios), `services` (llamadas a la API), `utils` (funciones puras de validación y formato) y `constants` (los textos). La separación obligatoria es **validadores puros ↔ hooks ↔ presentación**: la lógica de validar se prueba sin dibujar nada.

5. Formularios de entrada de datos (semana 05)

Se construyeron tres formularios, uno por entidad del enunciado: registrar usuario (RF-01), registrar ambiente (RF-02) y registrar solicitud de reserva (RF-03). Los tres se arman con los mismos componentes reutilizables (sección 7), la misma tarjeta (`FormularioTarjeta`) y el mismo hook de envío, de modo que se ven y se comportan igual.

5.1 Campos de cada formulario

Formulario	Campos	Detalles de uso
Registrar usuario	Nombre completo, correo institucional, rol, activo y contraseña inicial	Ejemplo del correo bajo el campo, contador de caracteres, política de la contraseña visible antes de escribir y botón «Mostrar»
Registrar ambiente	Nombre, tipo, capacidad, ubicación, estado y responsable	Tipo y estado como listas cerradas; el responsable se elige entre los usuarios con ese rol
Nueva reserva	Ambiente, fecha, hora de inicio, hora de fin y motivo	Ambiente en tarjetas (con capacidad y ubicación), horas como lista cerrada de 07:00 a 22:00 cada 30 minutos, atajos de duración (1, 2 o 3 h), aviso inmediato de disponibilidad y resumen en vivo de lo que se va a enviar

5.2 Nueva reserva (el formulario principal)

Los formularios comparten un mismo hook que guarda valores y errores, mueve el foco al primer campo inválido y evita el doble envío. La pantalla solo dibuja; la lógica no se repite en cada formulario.

Figura 8. Hook useFormulario: el estado de los formularios está separado de su presentación (frontend/src/hooks/useFormulario.js, líneas 22–45)

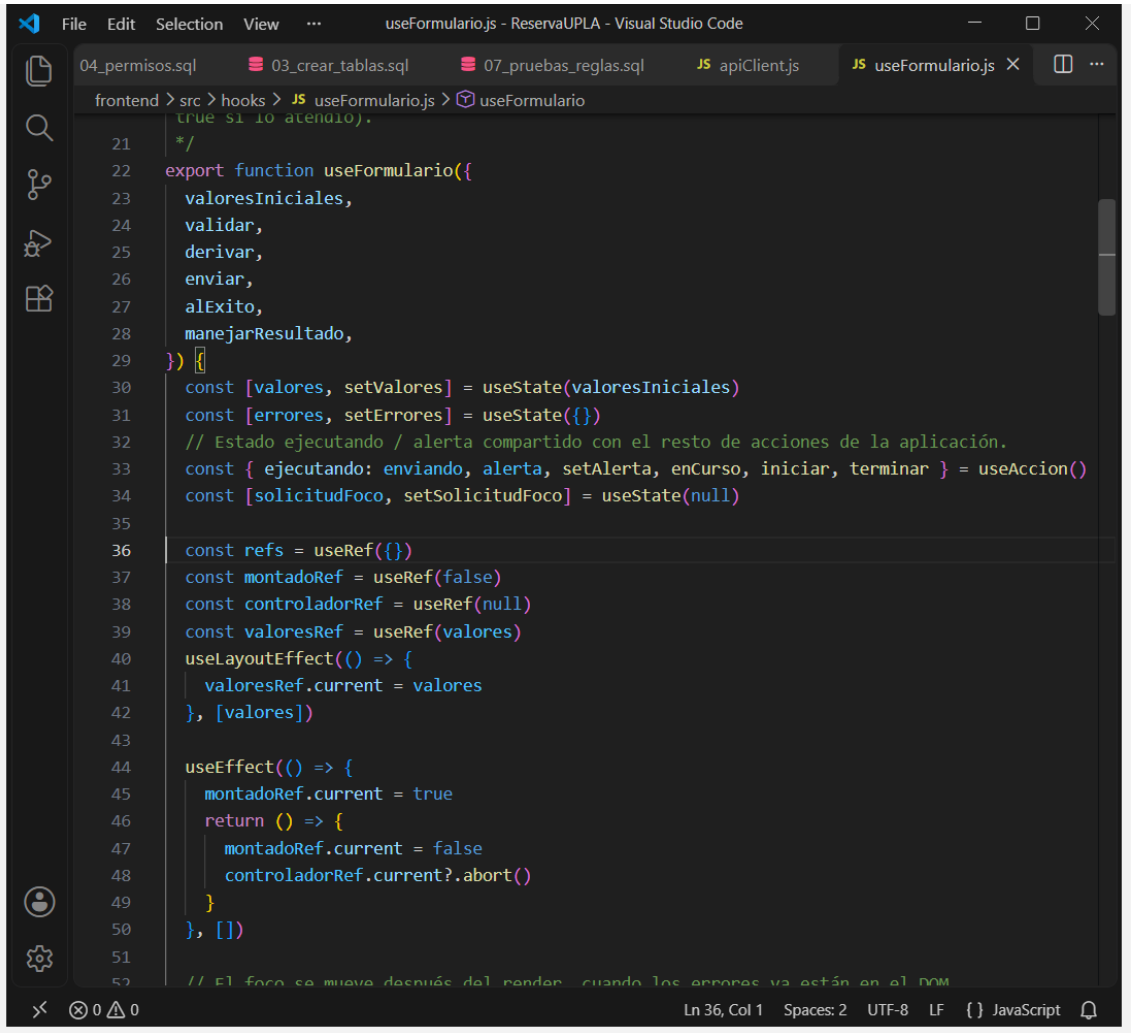
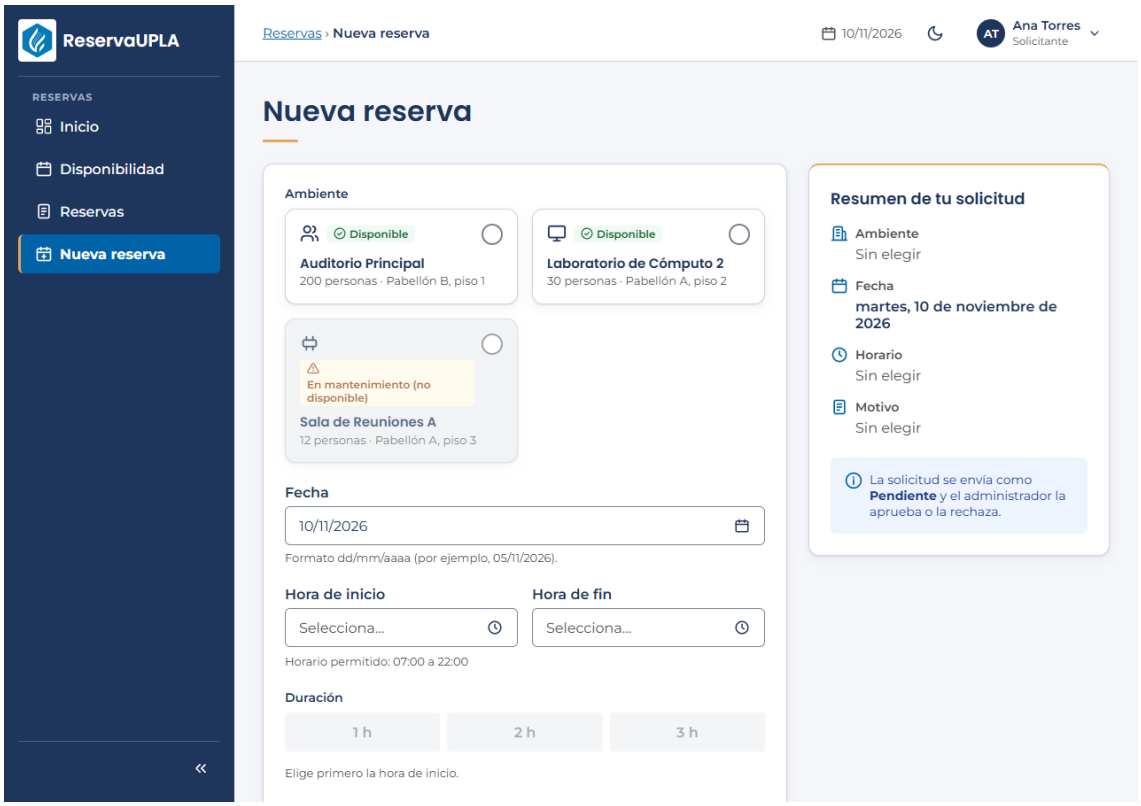
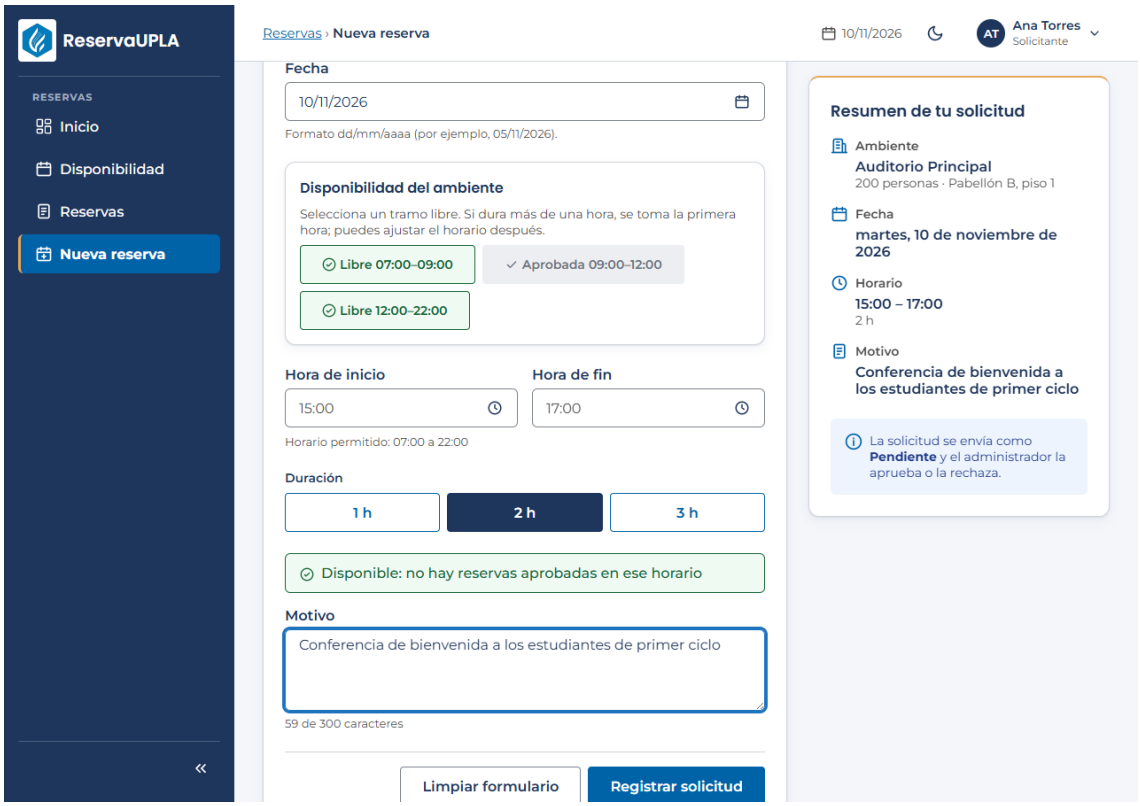


Figura 9. Formulario «Nueva reserva» recién abierto (Solicitante)



El ambiente que está en mantenimiento aparece deshabilitado y con su motivo visible. El selector es un grupo de opciones accesible (`role="radiogroup"`) que se maneja con las flechas del teclado (`frontend/src/components/ui/SelectorDeAmbientes.jsx:107`). Cuando se elige el ambiente, la fecha y las horas, el formulario consulta la disponibilidad y avisa antes de enviar.

Figura 10. Formulario con un ambiente elegido, horario y resumen en vivo



5.3 Registrar ambiente y registrar usuario

Figura 11. Formulario «Registrar ambiente» con un error de validación (Administrador)

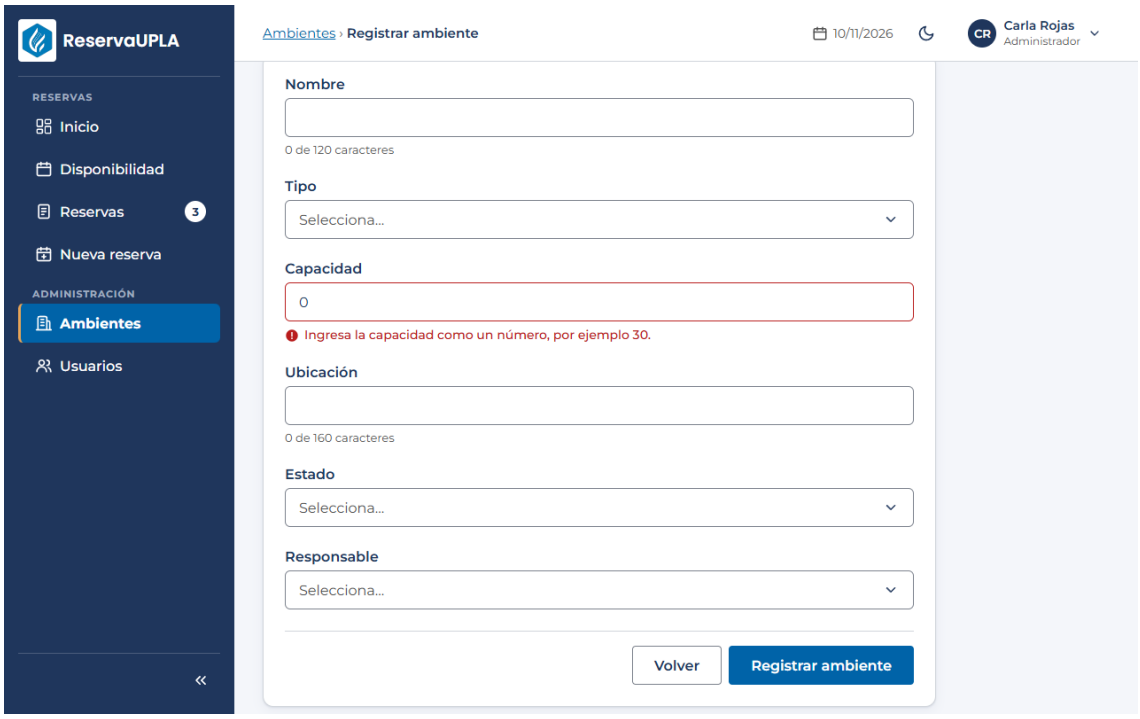
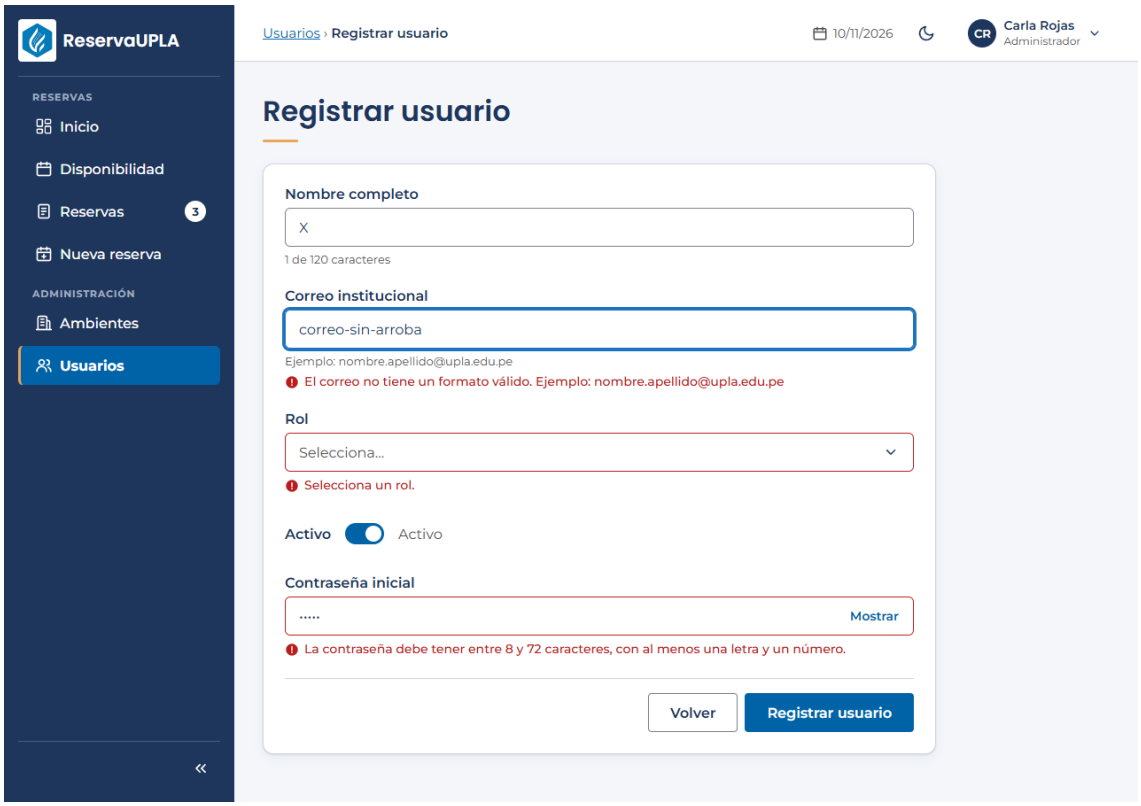


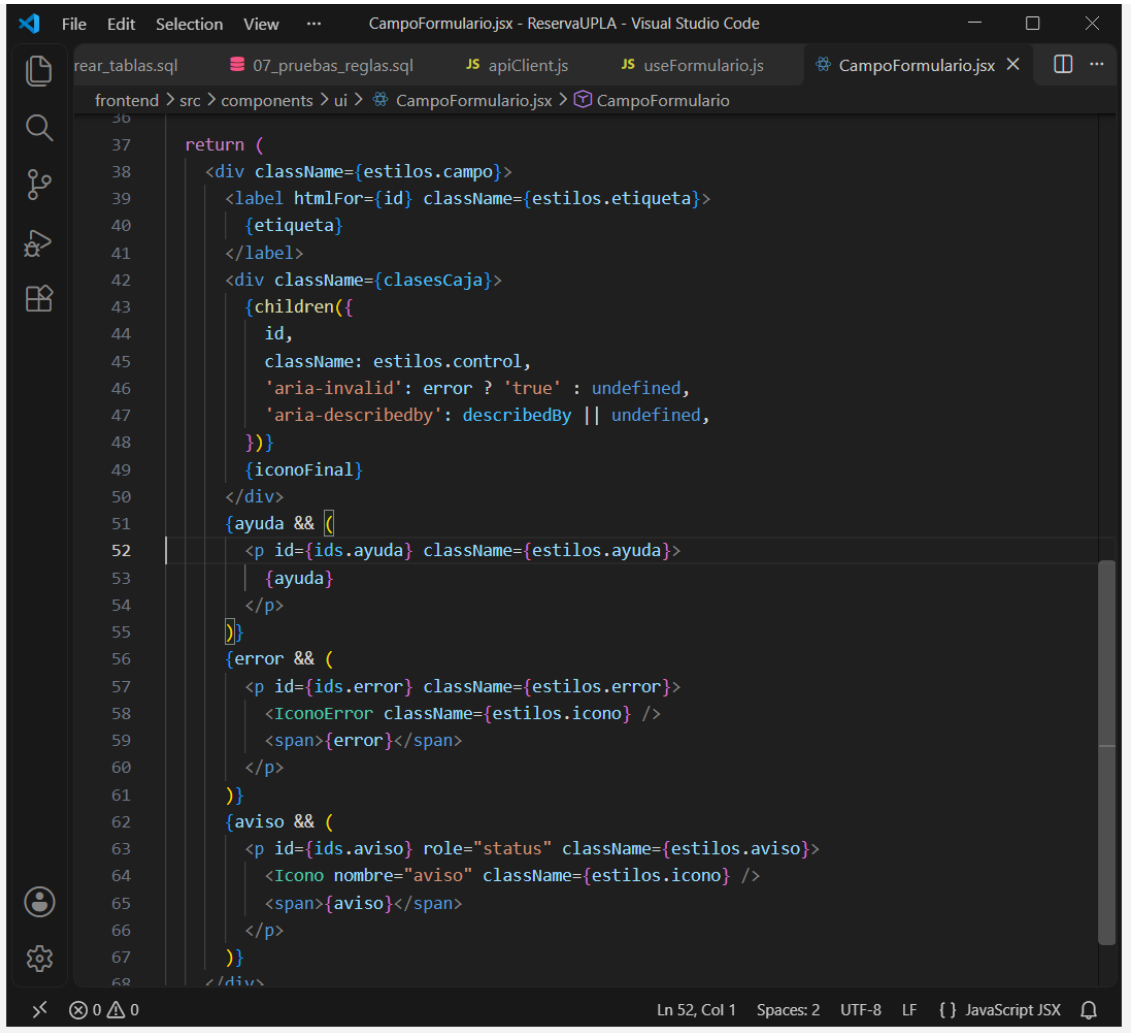
Figura 12. Formulario «Registrar usuario» con tres errores a la vez (Administrador)



5.4 Decisiones de usabilidad en los formularios

TextInput, SelectInput, DateInput y TimeInput usan esta misma estructura, por eso todos tienen la etiqueta visible, el error con icono junto al campo y los atributos de accesibilidad (aria-invalid, aria-describedby).

Figura 13. CampoFormulario: etiqueta, caja, ayuda, error y aviso de todos los campos (frontend/src/components/ui/CampoFormulario.jsx, líneas 38–65)



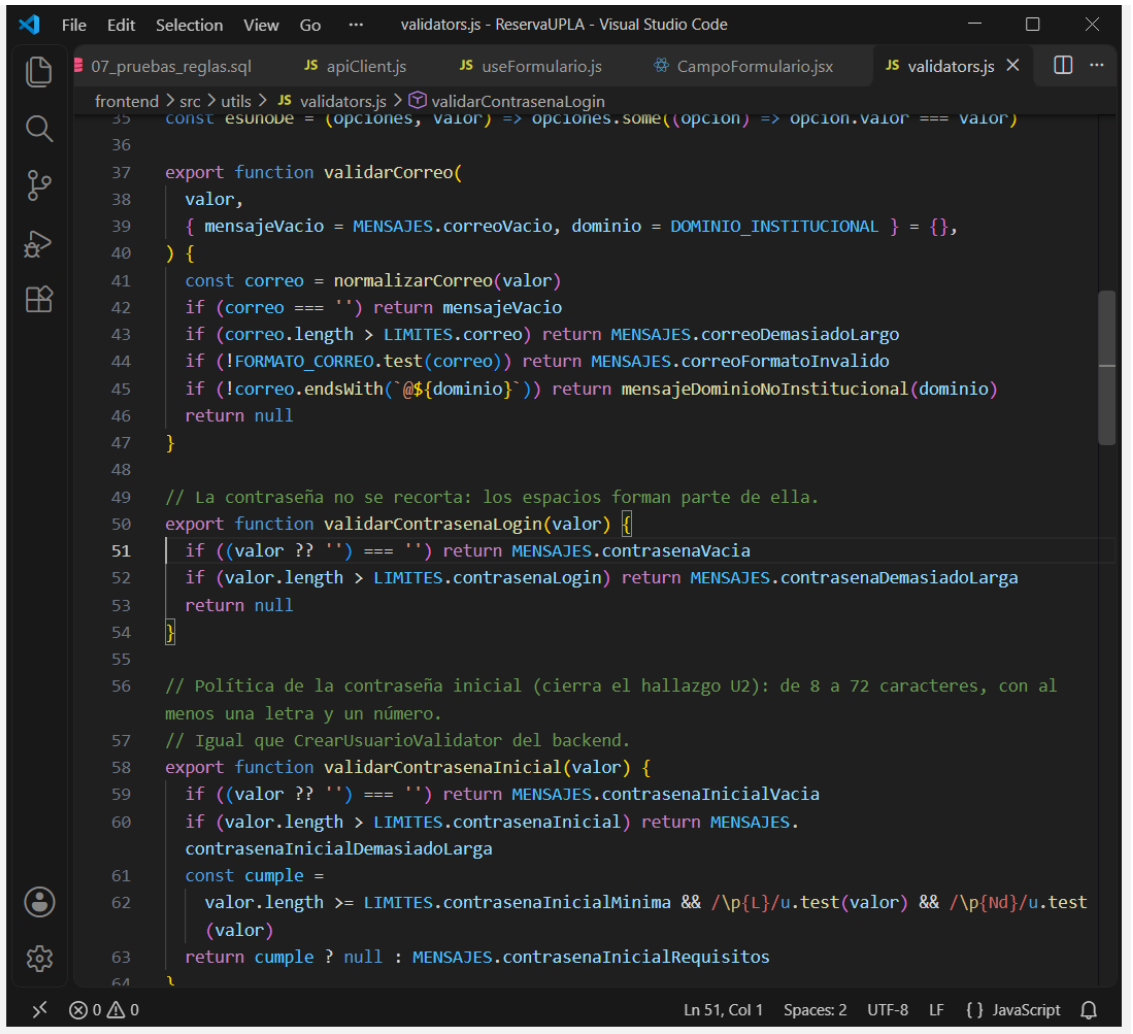
- Cada campo tiene su etiqueta visible siempre (no solo un texto de ayuda dentro del campo) y un ejemplo o una nota bajo él.
- Se valida al salir del campo y, una vez tocado, mientras se escribe: el error desaparece apenas se corrige. Al enviar, el foco va al primer campo con error.
- Los errores van junto al campo que los origina, con icono, texto y borde (nunca solo color).
- La hora es una lista cerrada, de modo que es imposible escribir una hora fuera de horario (RN-04) o un fin anterior al inicio (RN-03): se previene el error en lugar de solo avisarlo.
- Los botones dicen lo que hacen: «Registrar solicitud», «Registrar usuario», «Limpiar formulario» y «Volver».
- Todo el formulario se puede usar con el teclado, y en pantallas de 375 px los controles miden al menos 44 px.

6. Validaciones en frontend y backend (semana 06)

6.1 Tres capas, un mismo mensaje

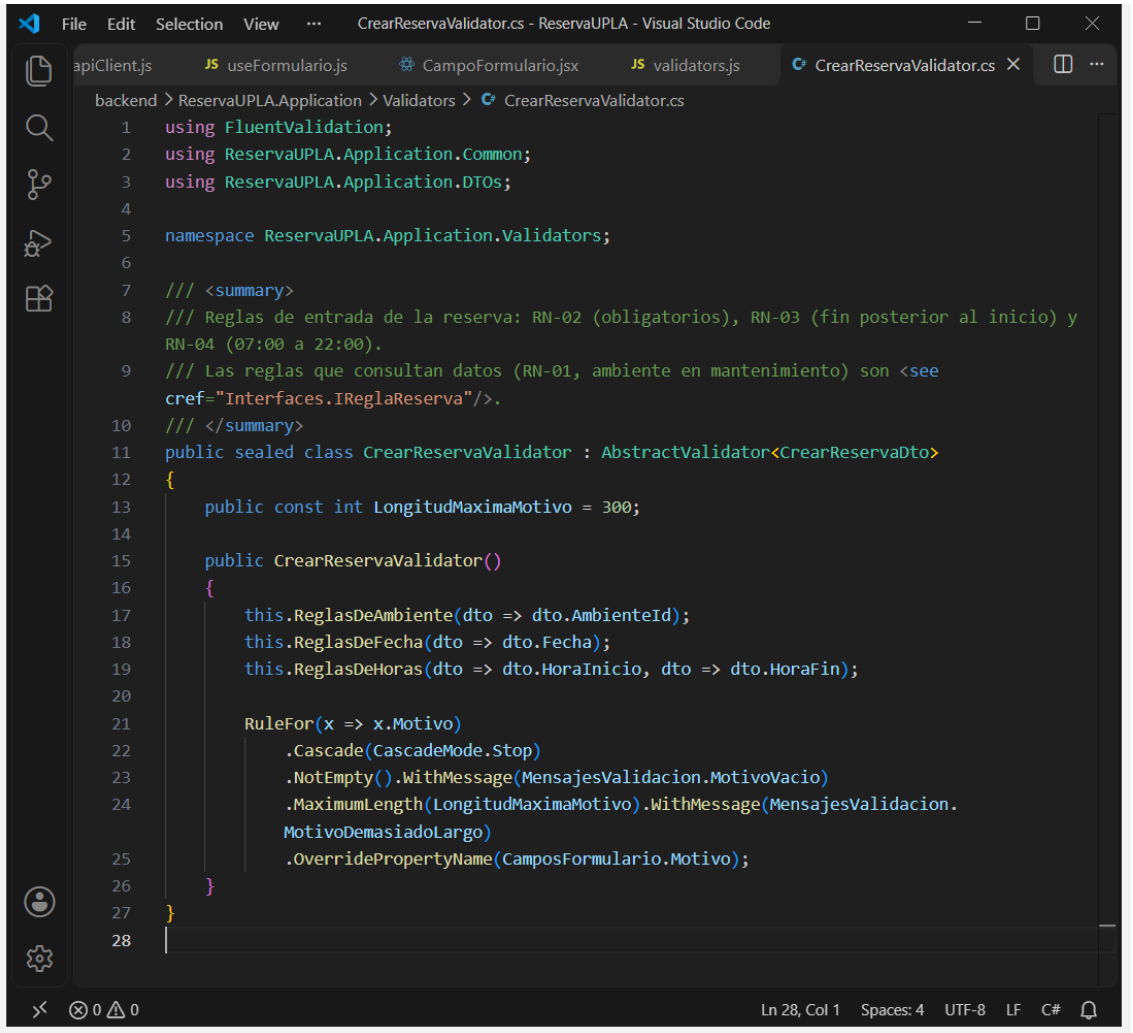
Normaliza el valor (recorta y pasa a minúsculas) y devuelve el mensaje de error o null. Al ser una función pura se prueba sin abrir el navegador.

Figura 14. Validación del correo en el frontend (función pura) (frontend/src/utils/validators.js, líneas 37–48)



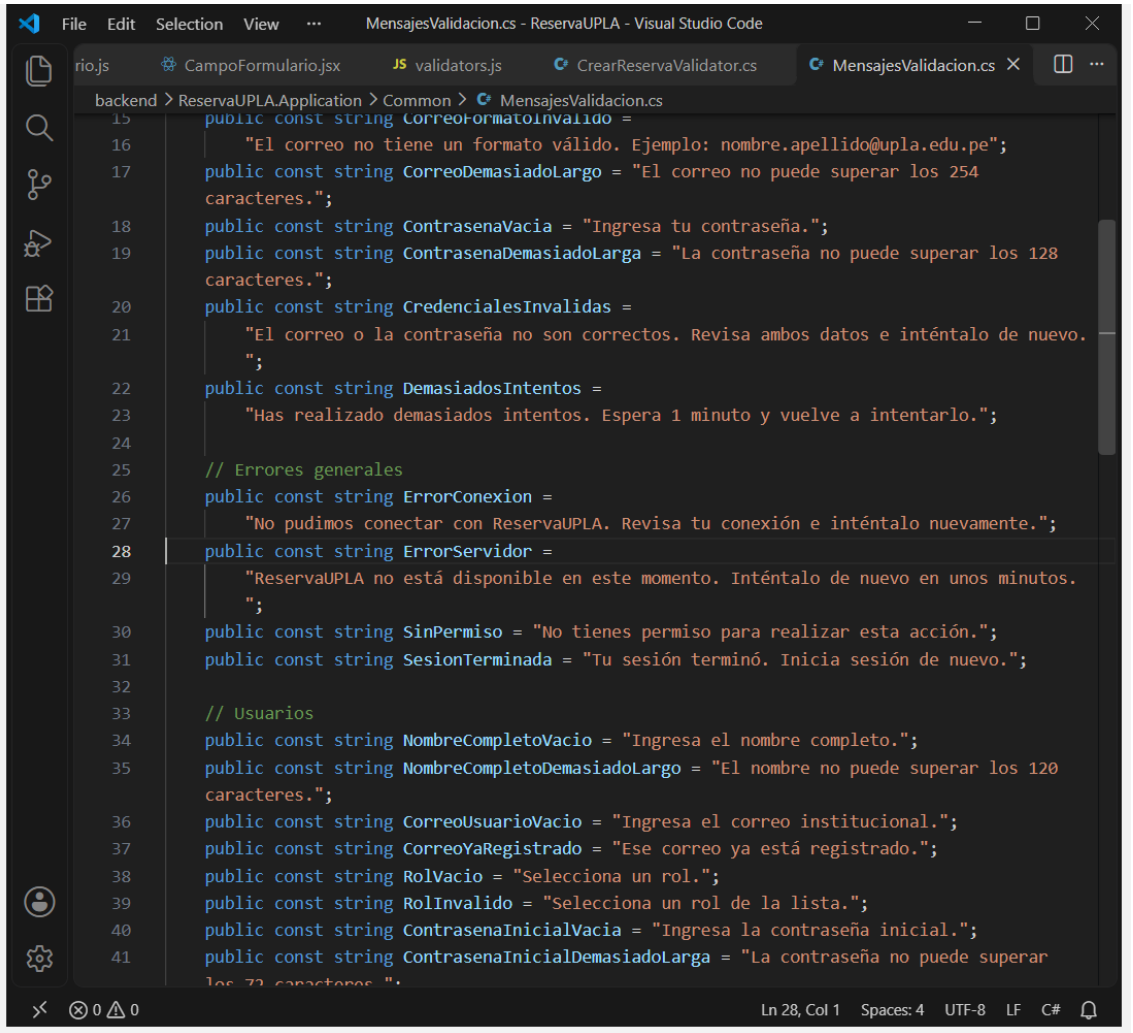
El backend repite las reglas del frontend, porque nunca se confía en lo que llega del cliente. Los mensajes salen de la clase MensajesValidacion, la misma fuente de textos que usa el frontend.

Figura 15. Validador de la reserva en el backend (FluentValidation) (backend/ReservaUPLA.Application/Validators/CrearReservaValidator.cs, líneas 15–26)



Todos los mensajes viven en un solo lugar, con texto que dice qué corregir. Cambiar un mensaje es cambiar una línea.

Figura 16. Mensajes de validación en una sola clase (backend/ReservaUPLA.Application/Common/MensajesValidacion.cs, líneas 14–19)

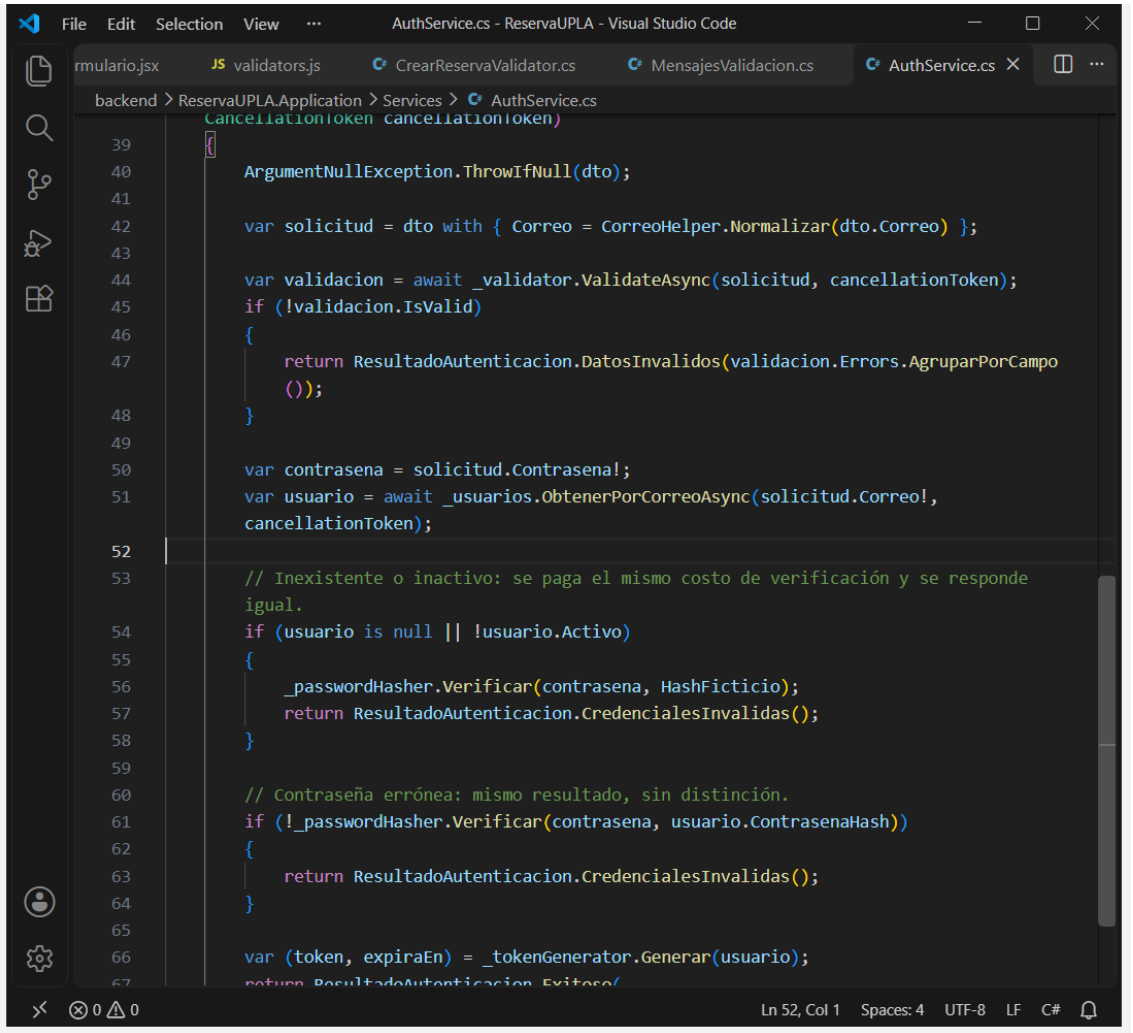


Una regla importante nunca se valida en un solo sitio. **El frontend ayuda** (avisa a tiempo y evita errores), **el backend decide** (es la única validación en la que se puede confiar, porque una llamada directa a la API se salta el formulario) y **la base de datos es la última barrera** (restricciones y disparadores). Los mensajes tienen una sola fuente en cada lado, y una prueba falla si un texto existe en un lado y no en el otro.

6.2 Dónde se aplica cada regla

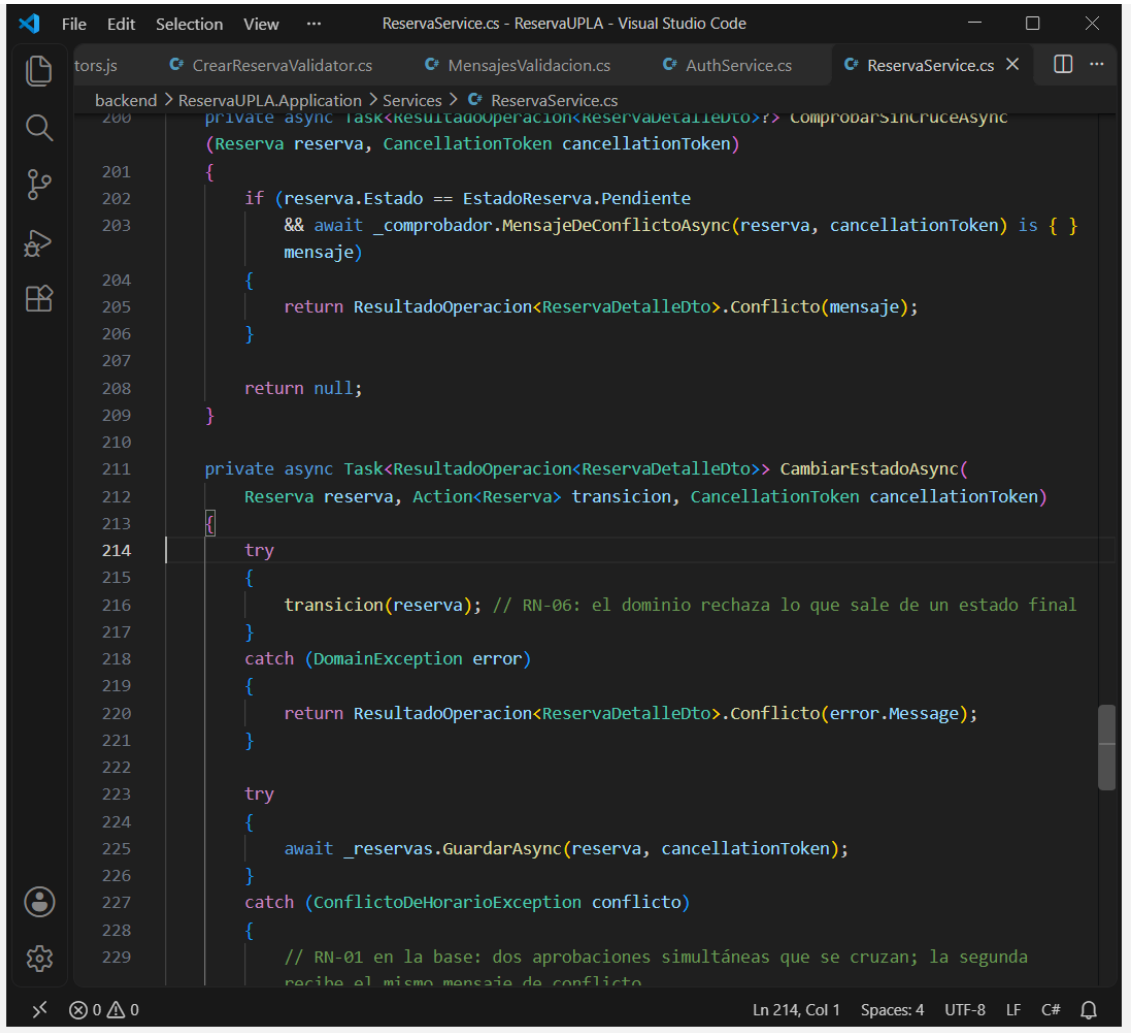
Aunque el usuario no exista, se verifica igual la contraseña contra un hash ficticio para que el tiempo de respuesta no revele si el correo está registrado. La respuesta es idéntica en los tres casos.

Figura 17. AuthService: el mismo resultado si el correo no existe, está inactivo o la clave es incorrecta (backend/ReservaUPLA.Application/Services/AuthService.cs, líneas 38–61)



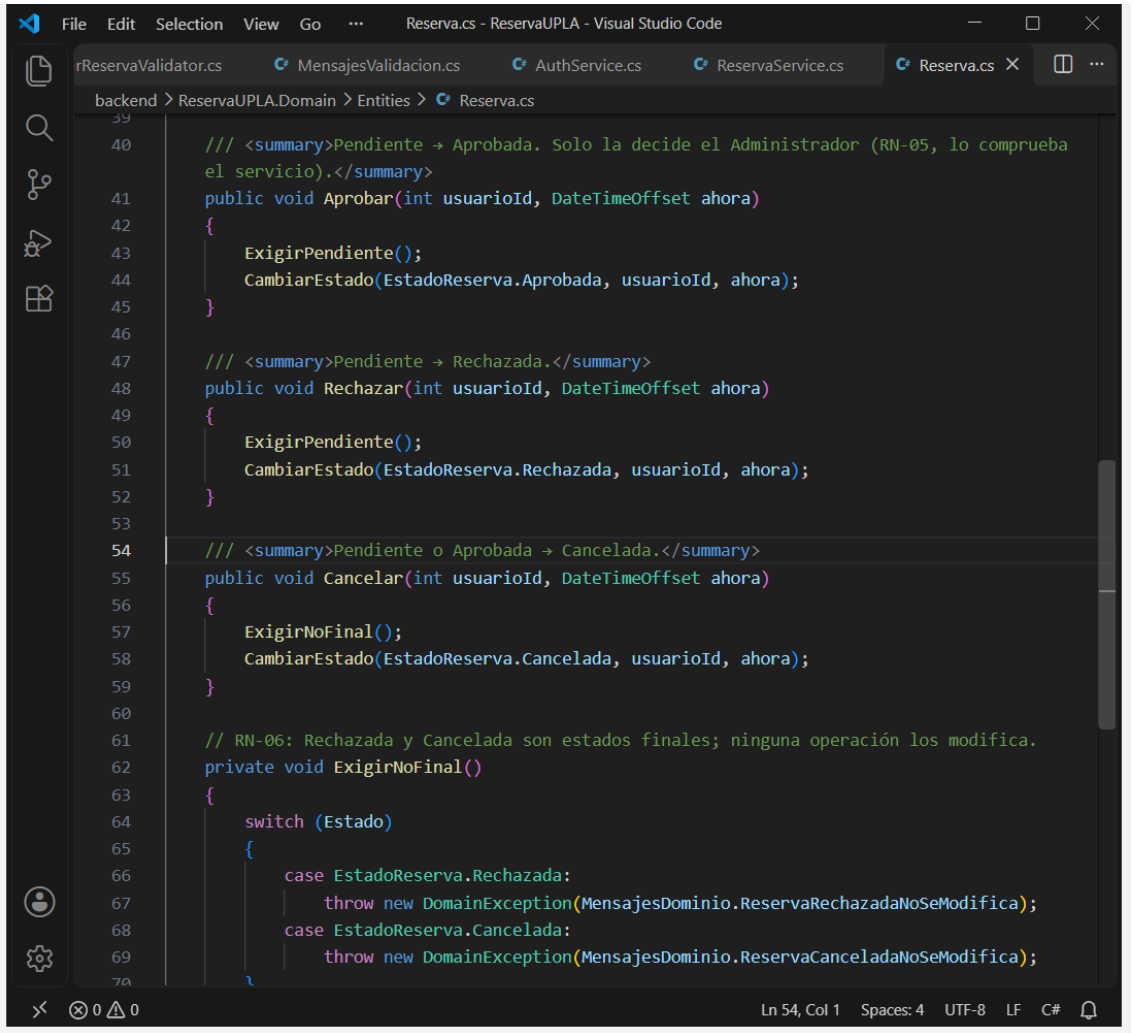
Las reglas que dependen de datos guardados las comprueba el servicio y devuelve un conflicto con un mensaje que dice con qué reserva choca.

Figura 18. ReservaService: comprobación de cruce antes de aprobar (RN-01) (backend/ReservaUPLA.Application/Services/ReservaService.cs, líneas 200–209)



El comportamiento está en la entidad y no en el servicio: una reserva que ya no está pendiente no se puede aprobar ni rechazar, y cada cambio deja registro de quién y cuándo.

Figura 19. Entidad Reserva: las transiciones de estado viven en el dominio (RN-06) (backend/ReservaUPLA.Domain/Entities/Reserva.cs, líneas 40–58)



Regla	Frontend	Backend	Base de datos
RN-01 una sola aprobada por ambiente, fecha y horario	Aviso inmediato de «ocupado» (`frontend/src/hooks/useDisponibilidad.js:26`)	Comprueba al registrar y al aprobar (`backend/ReservaUPLA.Application/Services/ReservaService.cs:200`)	Restricción de exclusión (`database/03_crear_tablas.sql:57`)
RN-02 datos obligatorios	Campos requeridos con mensaje (`frontend/src/utils/validators.js:135`)	Validadores de los DTO (`backend/ReservaUPLA.Application/Validators/CrearReservaValidator.cs:21`)	Columnas `NOT NULL`

Regla	Frontend	Backend	Base de datos
RN-03 fin mayor que inicio	La lista de fin solo ofrece horas posteriores (`frontend/src/utils/validators.js:127`)	`backend/ReservaUPLA.Application/Validators/ReglasDeCamposDeReserva.cs:42`	`database/03_crear_tablas.sql:53`
RN-04 entre 07:00 y 22:00	La lista cerrada solo ofrece de 07:00 a 22:00 cada 30 minutos	`backend/ReservaUPLA.Application/Validators/ReglasDeCamposDeReserva.cs:34`	`database/03_crear_tablas.sql:55`
RN-05 solo el administrador aprueba o rechaza	Los botones no aparecen a otros roles	`backend/ReservaUPLA.API/Controllers/ReservasController.cs:91–92` (responde 403)	No aplica (la identidad se comprueba en la API)
RN-06 una rechazada no se modifica	Acciones deshabilitadas con la razón a la vista	`backend/ReservaUPLA.Domain/Entities/Reserva.cs:41` (el dominio exige estado pendiente; responde 409)	Disparador `database/03_crear_tablas.sql:81`

6.3 Evidencia de las validaciones

Las figuras siguientes muestran las validaciones funcionando en la interfaz con el sistema real.

Figura 20. Enviar «Nueva reserva» vacía: un mensaje junto a cada campo y el foco en el primero

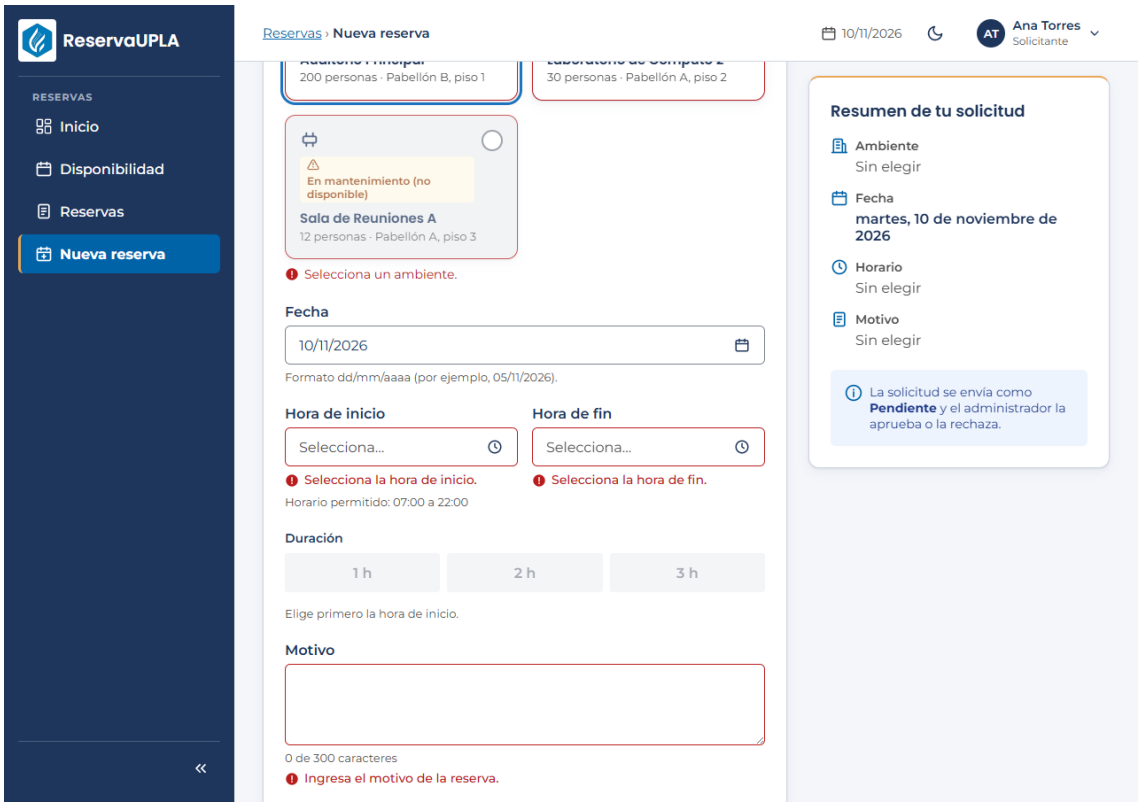


Figura 21. RN-01: aviso de disponibilidad con un tramo ocupado marcado «Choque»

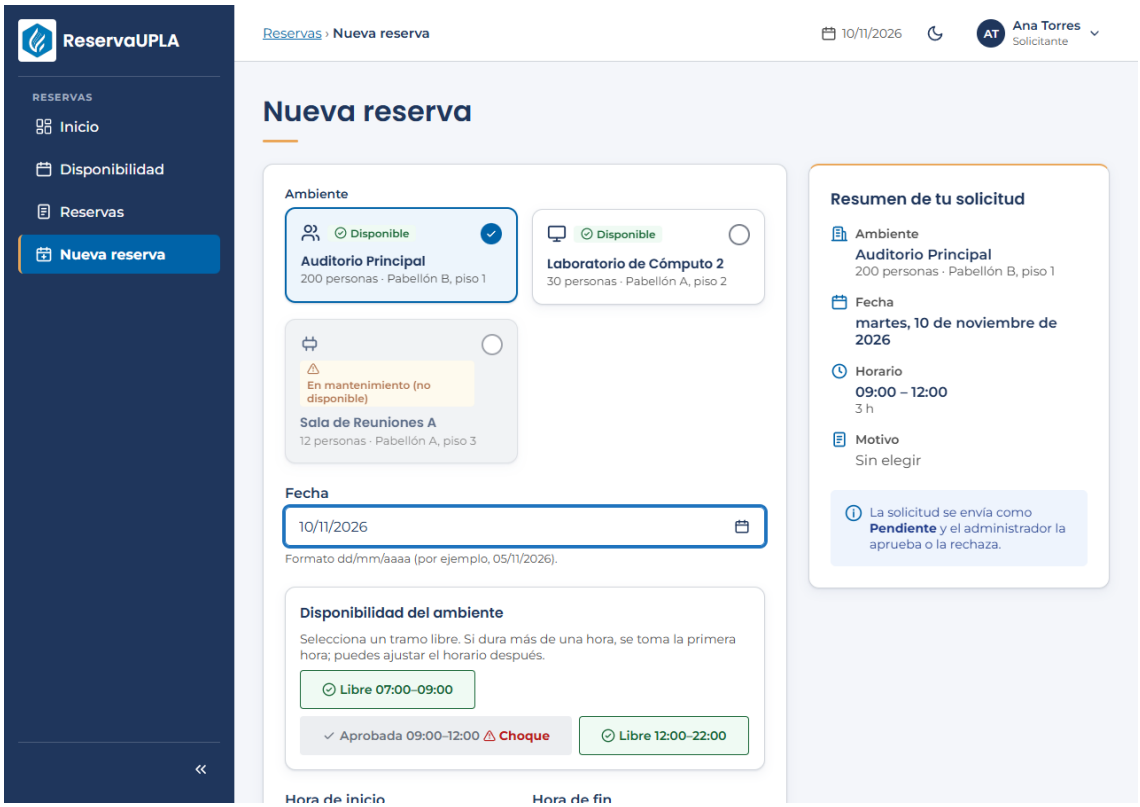
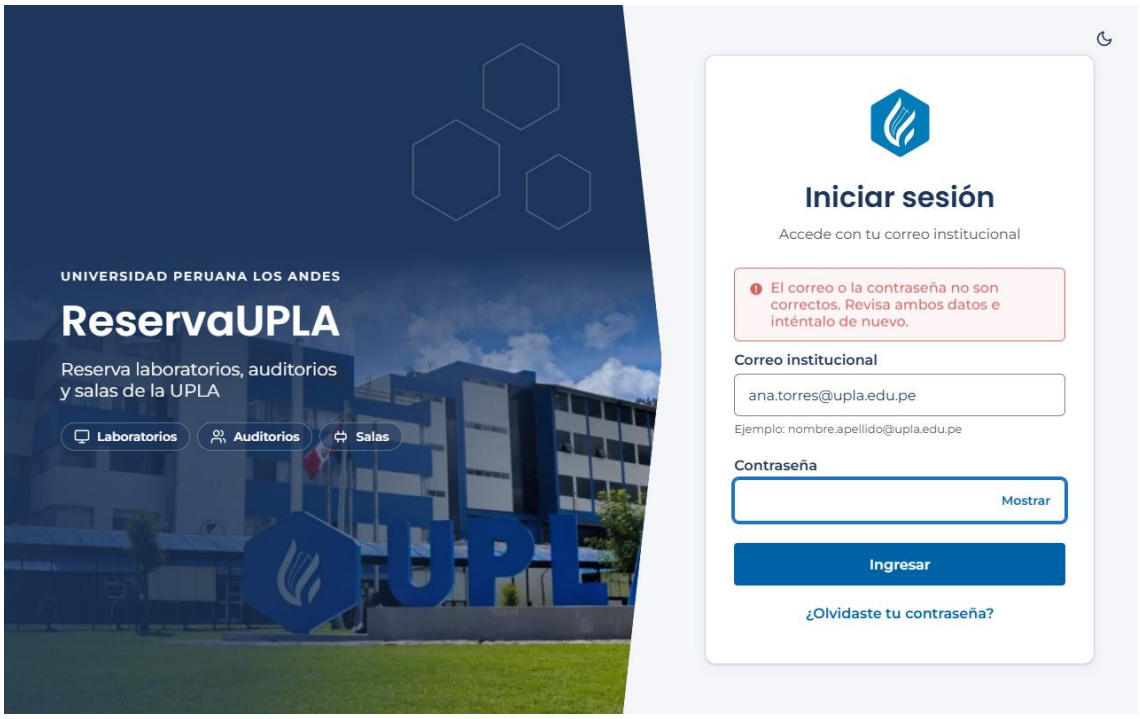


Figura 22. Credenciales incorrectas: mensaje genérico que no revela si el correo existe



6.4 Pruebas que rompen cada regla a propósito

Cada regla tiene, en cada capa, una prueba que intenta violarla. La matriz completa, con el nombre exacto de cada prueba, está en `docs/matriz-reglas.md`. Ejemplos de las más importantes:

- **RN-01 con concurrencia y base real:** `ConcurrenciaAprobacionTests.RN01_dos_aprobaciones_simultaneas_que_se_cruzan_dan_un_200_y_un_409_y_una_sola_Aprobada` lanza dos aprobaciones a la vez; la base deja pasar una y rechaza la otra, y la API responde 409 con la hora ocupada.
- **RN-05:** `ReservasEndpointsTests.RN05_solo_el_Administrador_aprueba_o_rechaza_los_demas_reciben_403` (HTTP 403) y las pruebas del frontend que verifican que un Solicitante no ve «Aprobar».
- **RN-06:** `ReservaTests.RN06_los_estados_finales_no_admiten_ninguna_transicion` en el dominio y `07_pruebas_reglas.sql` en la base (intenta pasar una rechazada a aprobada y exige que falle).
- **RN-03 y RN-04:** `ReservaServiceTests.RN03_hora_de_fin_no_posterior_al_inicio_se_rechaza`, `RN04_hora_de_inicio_antes_de_las_0700_se_rechaza` y los límites exactos 07:00 y 22:00 como válidos.

Un límite honesto: el frontend **ayuda, no decide**. Ocultar un botón no impide una llamada directa a la API, por eso cada regla se prueba también en el backend y en la base. Y el aviso de RN-01 del frontend no impide enviar la solicitud: entre el aviso y el envío otra persona puede haber aprobado una reserva, así que la decisión final siempre la toma el servidor (y, en última instancia, la base).

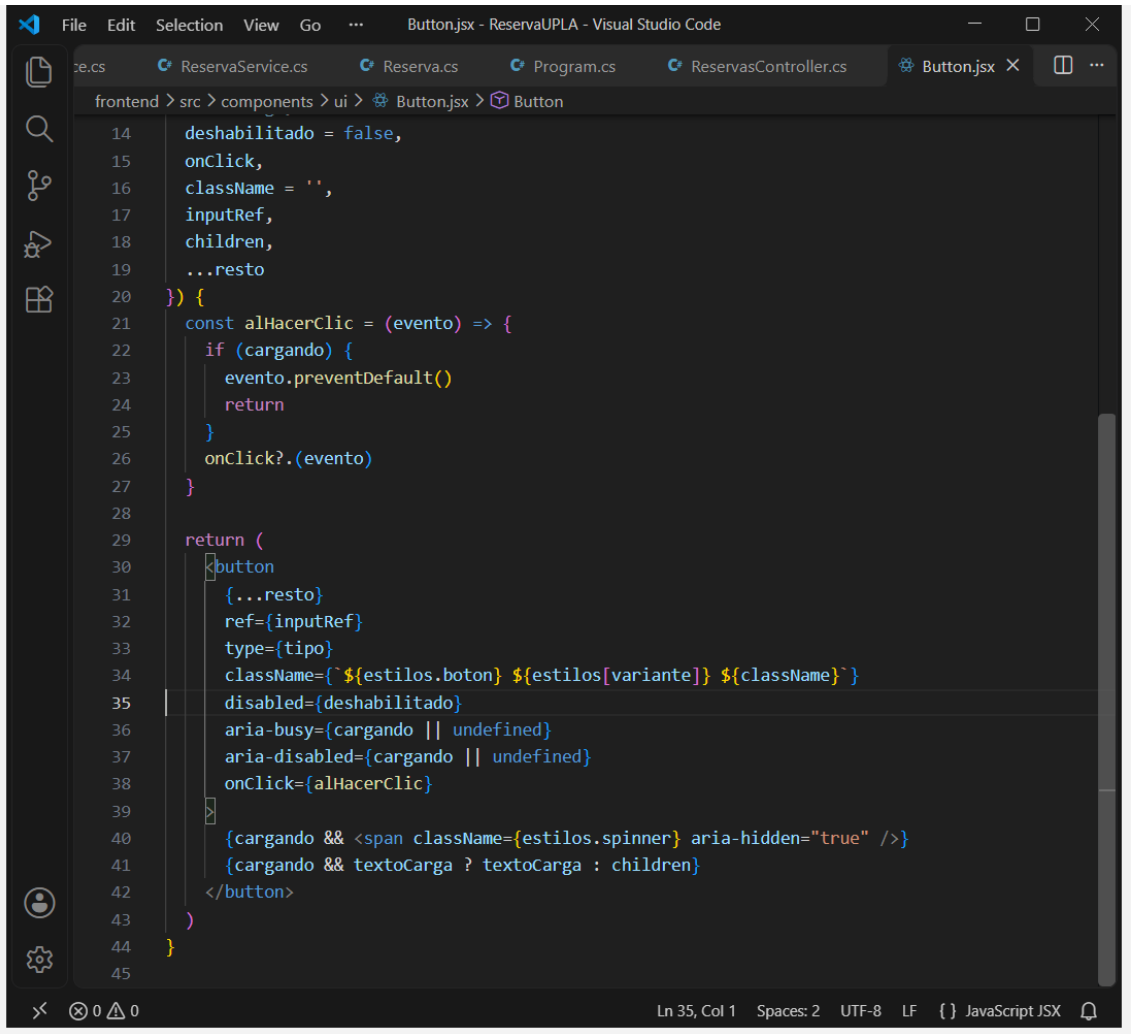
7. Reutilización de componentes (semana 07)

El caso pide componentes reutilizables para formularios y mensajes, y enumera nueve. Los nueve existen en `frontend/src/components/ui` y se usan en las pantallas; la columna de uso se **cuenta al generar este informe** recorriendo los `import` del código (no se escribe a mano).

7.1 Los nueve componentes mínimos

El botón único del sistema decide en un solo lugar cómo se ve cada variante, qué muestra al cargar y que no responde a más clics mientras procesa.

Figura 23. Button: bloqueo del doble clic mientras se procesa (frontend/src/components/ui/Button.jsx, líneas 21–28)



Componente	Qué resuelve	Archivos de la aplicación que lo usan	Ejemplos
`TextInput`	Campo de texto con etiqueta, ayuda, contador y error	7	`components/ui/DateInput.jsx`, `components/ui/PasswordInput.jsx`, `pages/AmbienteFormPage.jsx`
`SelectInput`	Lista cerrada con etiqueta y error (también la usan las horas)	5	`components/FiltrosDeReservas/FiltrosDeReservas.jsx`, `components/ui/TimeInput.jsx`, `pages/AmbienteFormPage.jsx`
`DateInput`	Fecha con formato dd/mm/aaaa y selector de calendario	3	`components/FiltrosDeReservas/FiltrosDeReservas.jsx`, `pages/DisponibilidadPage.jsx`, `pages/NuevaReservaPage.jsx`
`TimeInput`	Hora como lista cerrada de 07:00 a 22:00 cada 30 minutos	1	`pages/NuevaReservaPage.jsx`
`Button`	Botón único con variantes (primario, secundario, rechazar, enlace…) y estado de carga	27	`components/AtajosDeDuracion/AtajosDeDuracion.jsx`, `components/AvisoDeError/AvisoDeError.jsx`, `components/Avisos/Aviso.jsx`
`AlertMessage`	Mensajes de éxito, error, aviso e información, con icono y texto	12	`components/AvisoDeError/AvisoDeError.jsx`, `components/Disponibilidad/FranjaNuevaReserva.jsx`, `components/FormularioTarjeta/FormularioTarjeta.jsx`
`ModalConfirmacion`	Confirmación con foco atrapado, Esc para salir y el verbo exacto en el botón	1	`components/ModalDeReserva/ModalDeReserva.jsx`
`TableList`	Tabla con encabezado, columnas y filas para Reservas, Ambientes, Usuarios y Dashboard	5	`pages/AmbientesPage.jsx`, `pages/ListadoPage.jsx`, `pages/MiAmbientePage.jsx`
`CardIndicador`	Tarjeta de indicador con icono, número y descripción	2	`components/GrupoDeIndicadores/GrupoDeIndicadores.jsx`, `pages/InicioResponsable.jsx`

7.2 Más allá de los nueve

Al reutilizar se fueron extrayendo piezas que se repetían: `FormularioTarjeta` (la misma tarjeta en los tres formularios), `GrupoDeIndicadores` y `PageHeader` (cabecera común de todas las pantallas), `EstadoVacio` y `EsqueletoDeLista` (estado vacío y carga), `Insignia` (estado con icono y texto), `PasswordInput`, `Avatar` y `Paginacion`. En lógica, `useFormulario` (campos, validación y envío), `useAccion` (estado «ejecutando» y avisos de éxito o error) y `mensajesDeResultado.js` (la traducción de cada código HTTP a un mensaje).

7.3 Cómo se evita que la duplicación vuelva

- **Medición de duplicación.** Con jscpd 4.0.5, perfil de 4 líneas y 30 tokens, la duplicación pasó de **1,38 %** (antes de la semana 07) a **0,93 %** (con todas las pantallas terminadas). El frontend quedó en 0 bloques duplicados; lo que queda es de C# y son encabezados de archivo (`using`, `namespace`) que el lenguaje obliga a repetir. Los informes completos están en `docs/evidencias/fase7-duplicacion-*.txt` y `docs/reutilizacion.md`.
- **Prueba de arquitectura.** `frontend/src/arquitectura.test.js:31` falla si una pantalla escribe `<input>`, `<select>`, `<textarea>` o `<button>` en lugar de usar los componentes de `components/ui`. Se comprobó que falla poniendo un `<button>` suelto.
- **Prueba de mensajes.** `messages.test.js` falla si un mensaje de la interfaz no existe idéntico en el backend (o al revés).
- **Colores y formas en un solo archivo.** `tokens.css` es la única fuente de colores, radios y sombras, y `identidad.test.js` falla si una hoja de estilo escribe un color suelto.

8. Refinamiento de código (semana 08)

Refinar es mejorar la estructura sin cambiar lo que el sistema hace, y comprobarlo con pruebas. Se hizo en dos momentos: la reorganización de servicios y formularios de la semana 08, y las correcciones que salieron de probar el sistema con datos reales al final del proyecto.

8.1 Refactorizaciones

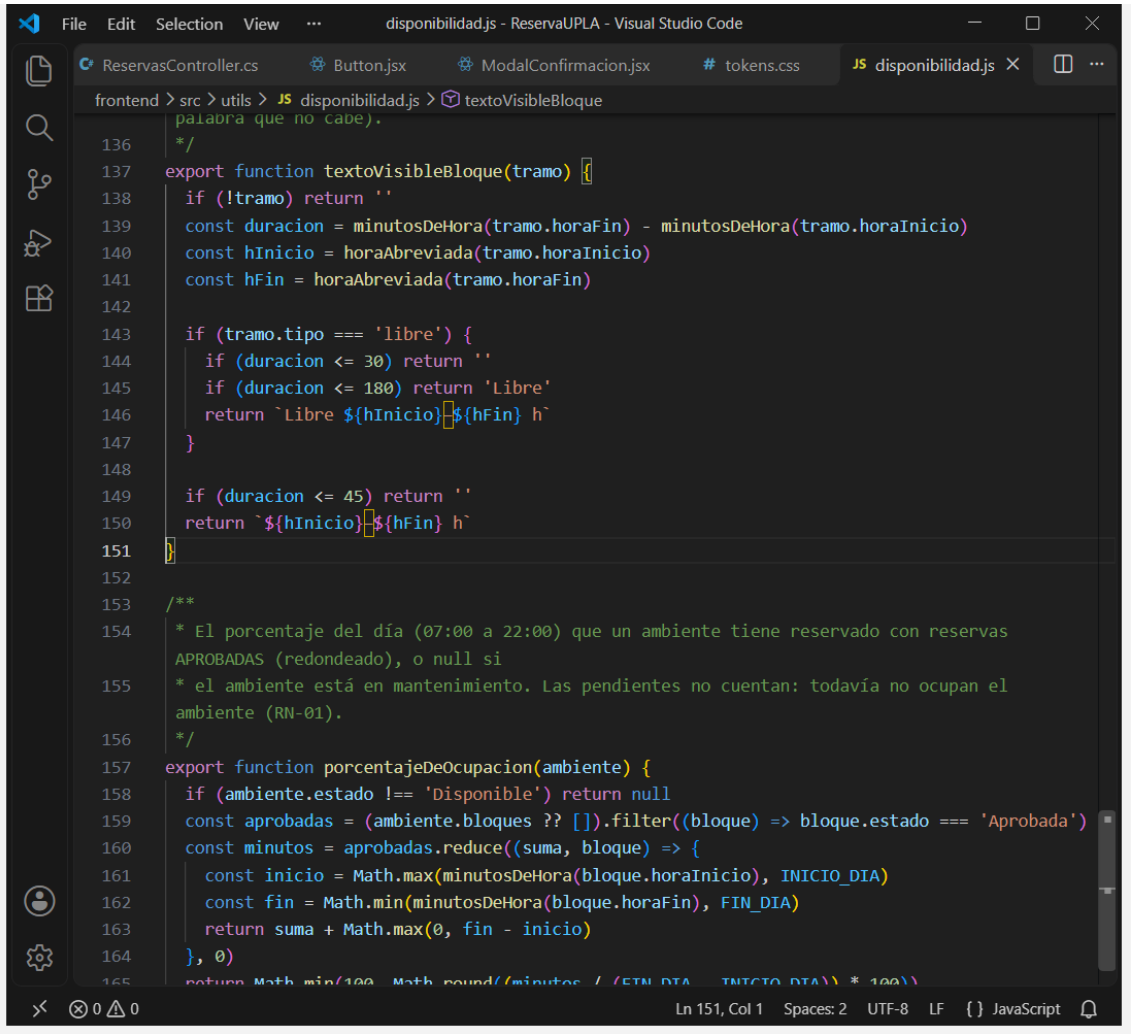
Zona	Antes	Ahora
Backend: cambiar de estado	Aprobar, rechazar y cancelar repetían buscar, comprobar permiso, comprobar estado y guardar	Un solo método `TransicionarAsync` con la comprobación y la transición como parámetros (`backend/ReservaUPLA.Application/Services/ReservaService.cs:173`)
Backend: de resultado a HTTP	Cada acción de los controladores traducía el resultado a mano	`ControllerBaseExtensions` (`Traducir`, `Creado`, `DatosInvalidos`): los controladores quedaron delgados
Backend: contraseña	La regla de 8 a 72 caracteres con letra y número estaba en dos validadores	Una sola `ReglasDeContrasena` usada por el alta de usuario y el login

Zona	Antes	Ahora
Frontend: mensajes de error de la API	Cada hook y página traducía 400, 401, 403, 404, 409, 429, 5xx y falta de red por su cuenta	`services/mensajesDeResultado.js`
Frontend: formularios	Cada formulario armaba sus campos y su envío	`useFormulario` y `useFormularioDeRegistro`
Frontend: pantallas de lista	Usuarios y Ambientes repetían la misma estructura	`ListadoPage` compartida

8.2 Defectos hallados al probar con datos reales

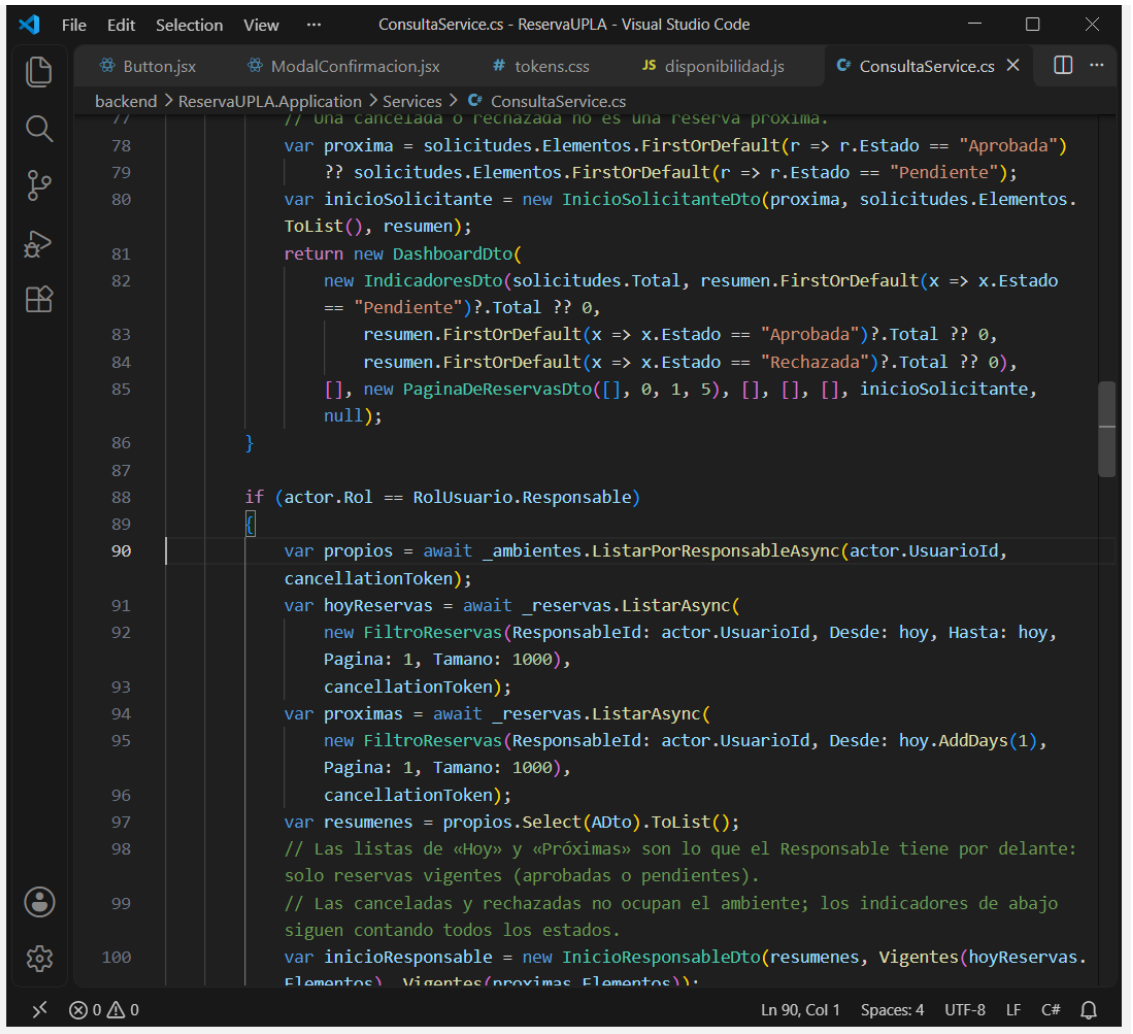
Al probar con datos reales, los textos de los tramos cortos aparecían recortados. Esta función decide qué texto cabe según la duración, y quedó cubierta con pruebas.

Figura 24. Corrección de un defecto real: el texto de cada tramo depende de su ancho (frontend/src/utils/disponibilidad.js, líneas 137–150)



Mirando el sistema con datos reales apareció una reserva cancelada como «próxima». Ahora se prefiere la aprobada y, si no hay, la pendiente.

Figura 25. Corrección de un defecto real: «Tu próxima reserva» ya no muestra una cancelada (backend/ReservaUPLA.Application/Services/ConsultaService.cs, líneas 76–80)



Las pruebas unitarias pasaban, y aun así, al levantar el sistema completo con los datos de demostración aparecieron defectos que ninguna prueba cubría. Cada uno se corrigió y quedó una prueba que lo impide:

Defecto observado en pantalla	Causa	Corrección y prueba
En el Inicio del Solicitante, «Tu próxima reserva» mostraba una reserva **cancelada**	El backend tomaba la primera reserva del solicitante sin mirar su estado	Ahora elige la aprobada más cercana y, si no hay, la pendiente (`ConsultaService`); dos pruebas nuevas
El Inicio del Responsable listaba reservas canceladas y rechazadas como «próximas»	No se filtraba por estado vigente	Solo aprobadas y pendientes; prueba nueva

Defecto observado en pantalla	Causa	Corrección y prueba
Con el menú lateral contraído, la columna quedaba de 80 px de alto con los iconos recortados y una barra de desplazamiento	Un `flex: 0 0 72px` dentro de una columna se aplicaba al alto y no al ancho	Se quitó la regla; prueba de CSS que impide que vuelva
Un tramo de la línea de tiempo se leía «9–12 h · Aproba…» (texto cortado)	La regla de texto decidía por duración y no por el ancho disponible	Los tramos muestran solo el horario; el estado lo dice el icono; pruebas de borde y comprobación en navegador
En «Próximas reservas» del Responsable, el botón «Ver detalle» caía en una fila aparte	La fila tenía seis datos en una rejilla de cinco columnas	Rejilla de seis columnas

Todos los hallazgos del proyecto, con su corrección y la prueba que los cubre, están en `docs/bitacora-evaluacion.md`. La lección del refinamiento: una prueba unitaria demuestra lo que alguien pensó probar; probar el sistema completo con datos reales encuentra lo que nadie pensó.

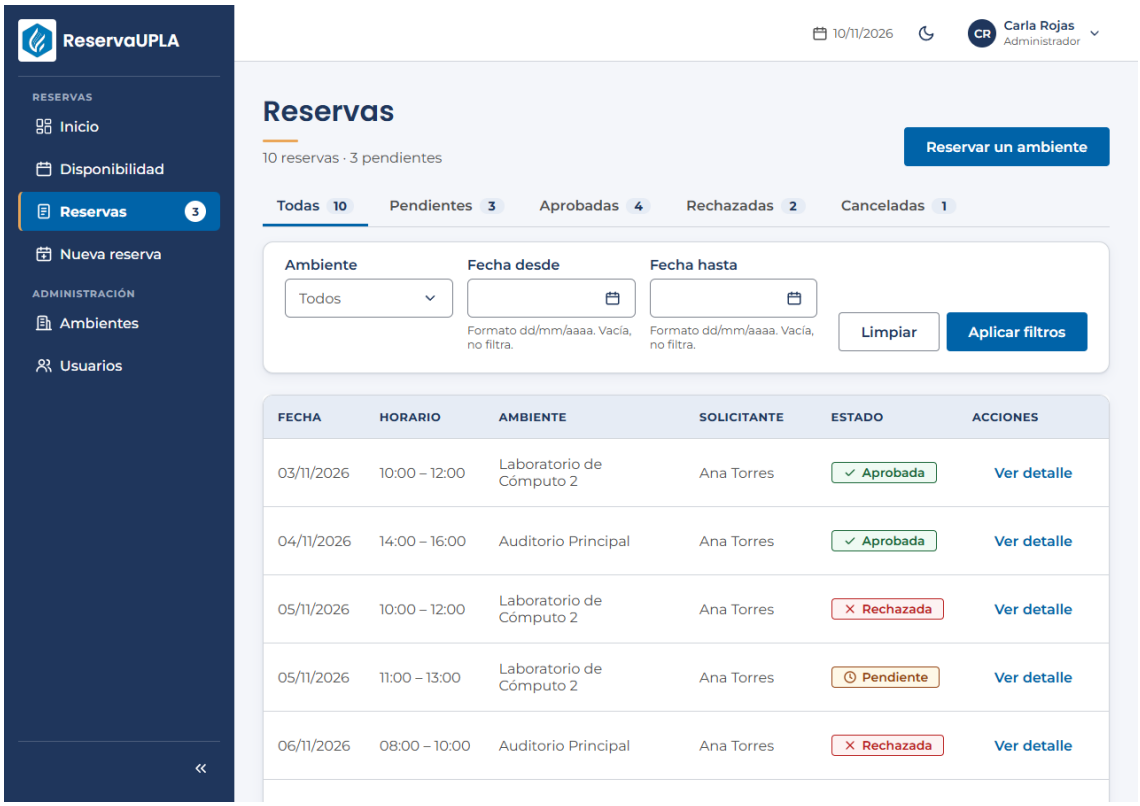
9. Módulos de salida de datos y dashboard (semana 09)

Los módulos de salida muestran lo que el sistema sabe: el listado de reservas (RF-05), el detalle con su historial y el cambio de estado (RF-06 y RF-07) y el dashboard (RF-08). Cada rol ve solo lo que le corresponde: el Solicitante sus reservas, el Responsable las de su ambiente y el Administrador todas.

9.1 Listado de reservas (RF-05)

El listado tiene pestañas por estado con su cantidad, filtros por ambiente y rango de fechas, tabla con insignias de estado (icono más texto, nunca solo color) y paginación del lado del servidor. El alcance por rol se aplica en la consulta a la base, no en pantalla: lo que no le corresponde al rol nunca sale de la base de datos.

Figura 26. Listado de reservas con pestañas por estado, filtros y tabla (Administrador)

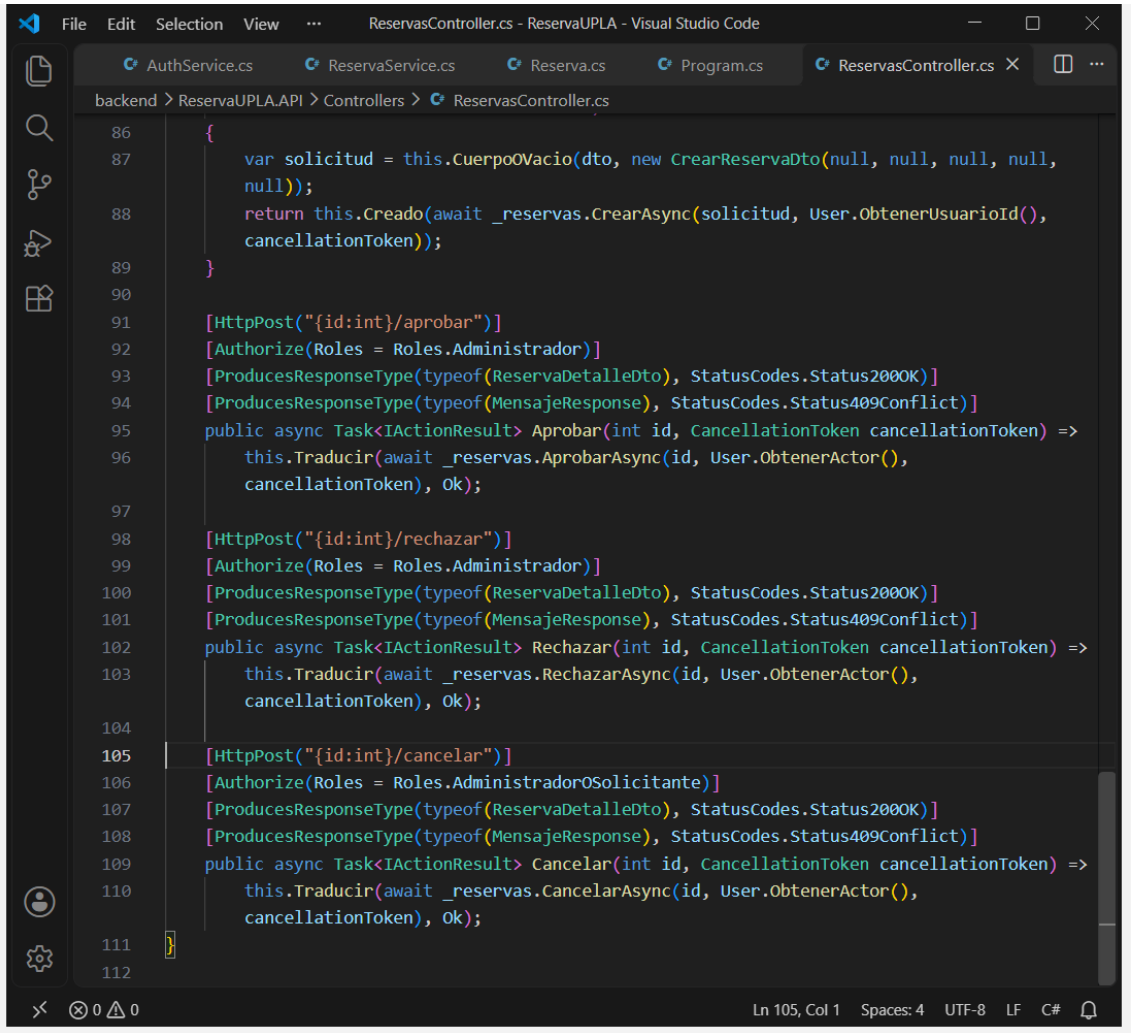


Implementación: `frontend/src/pages/ReservasPage.jsx:31` (pantalla) y `backend/ReservaUPLA.Infrastructure/Repositories/ReservaRepository.cs:27` (consulta con el alcance del rol).

9.2 Detalle y cambio de estado (RF-06 y RF-07)

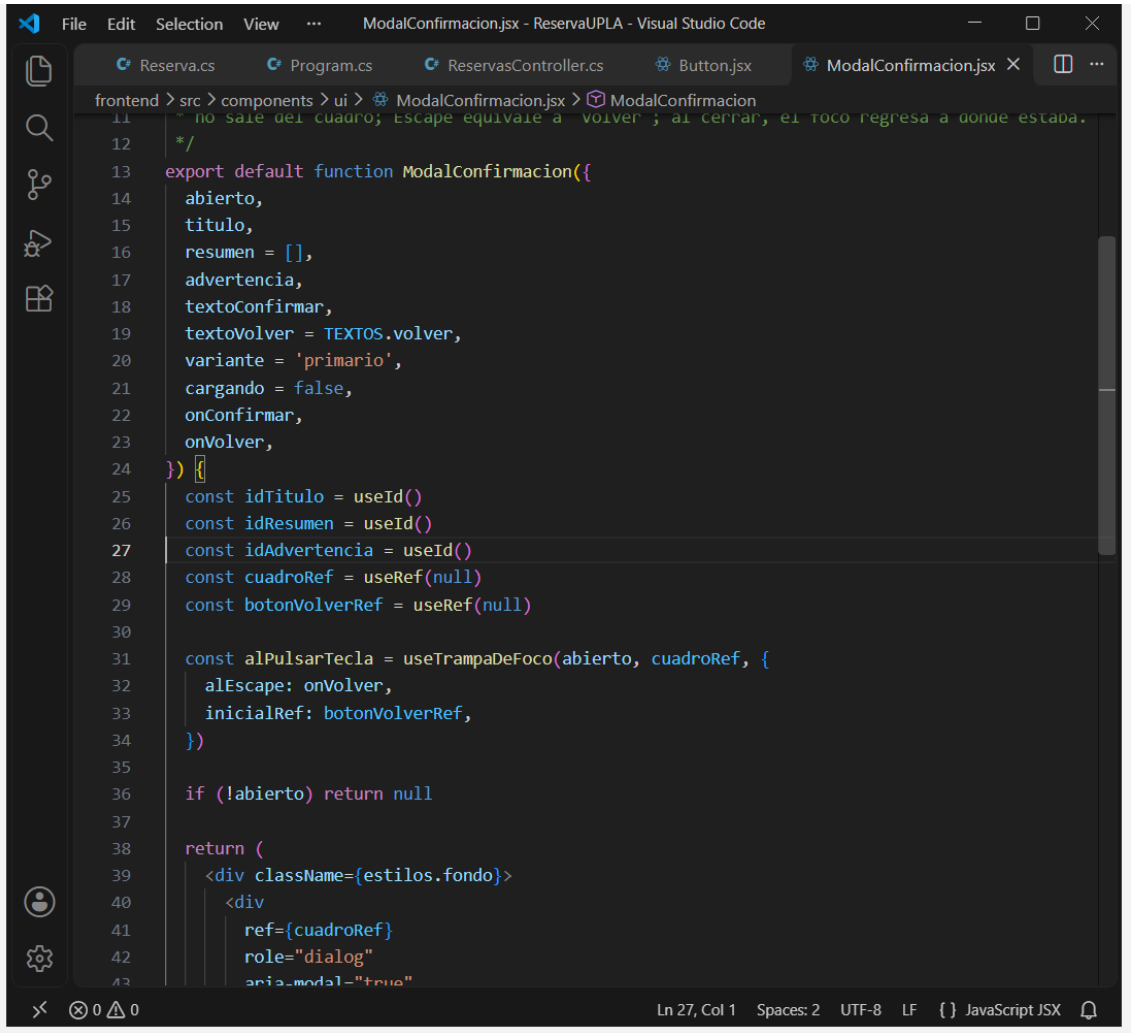
El controlador no tiene lógica de negocio: solo exige el rol de Administrador (RN-05), llama al servicio y convierte su resultado en una respuesta HTTP.

Figura 27. Controlador delgado: autoriza, delega en el servicio y traduce el resultado a HTTP (backend/ReservaUPLA.API/Controllers/ReservasController.cs, líneas 91–96)



Aprobar, rechazar y cancelar usan el mismo modal; solo cambian el título, el resumen y el texto del botón. El foco empieza en «Volver», la opción segura.

Figura 28. ModalConfirmacion: props del cuadro de confirmación reutilizable (frontend/src/components/ui/ModalConfirmacion.jsx, líneas 13–25)



El detalle muestra el ambiente, el horario en letras con su duración, el estado destacado y el historial como una línea de tiempo: quién hizo cada cambio y cuándo. Las acciones dependen del rol y del estado. Aprobar y rechazar piden confirmación con el verbo exacto («Sí, aprobar»), y si aprobar causaría un choque, el botón queda deshabilitado y una tarjeta explica el choque y enlaza a la reserva que lo causa.

Figura 29. Detalle de una reserva pendiente con su historial y acciones (Administrador)

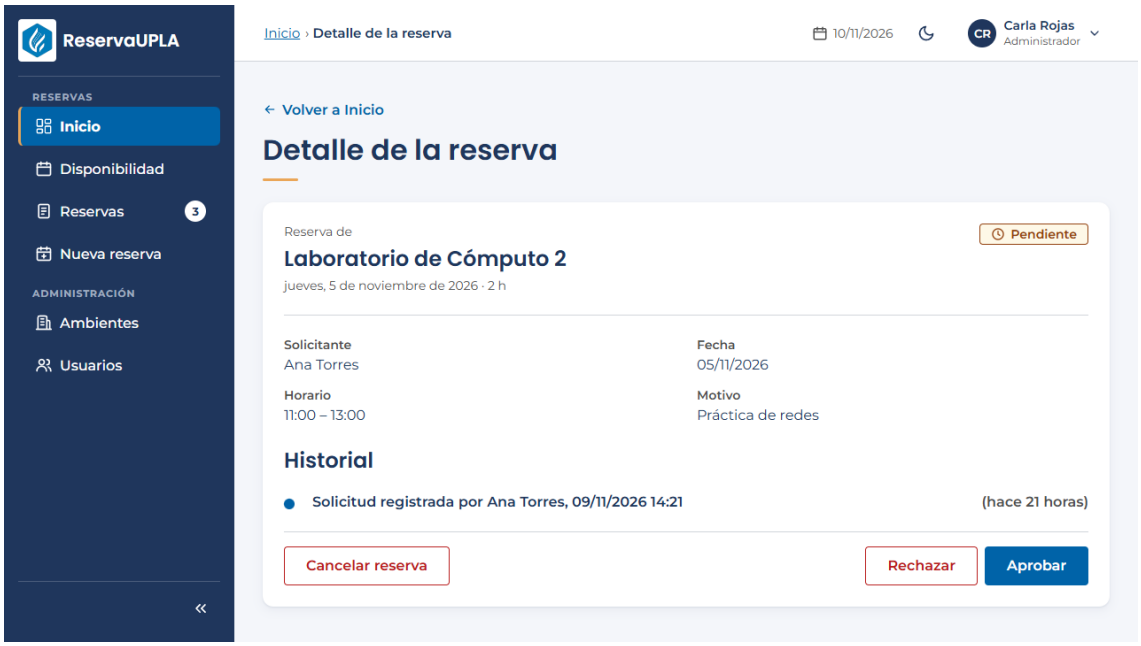


Figura 30. Reserva pendiente que choca con una aprobada: «Aprobar» queda deshabilitado y se explica por qué

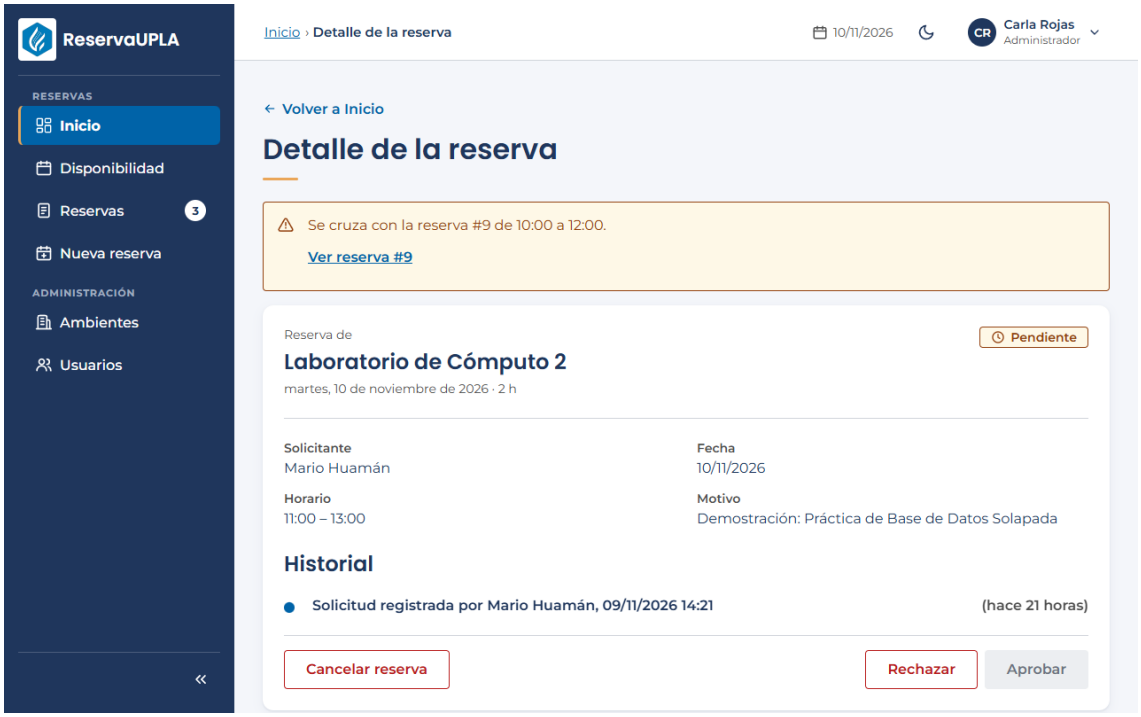
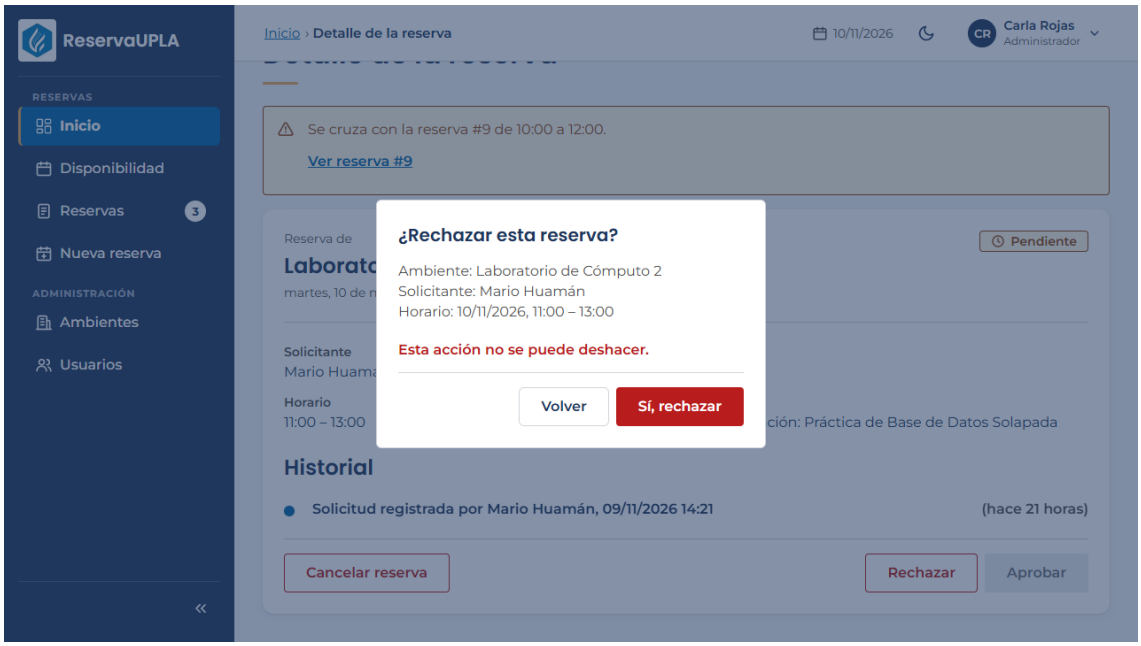


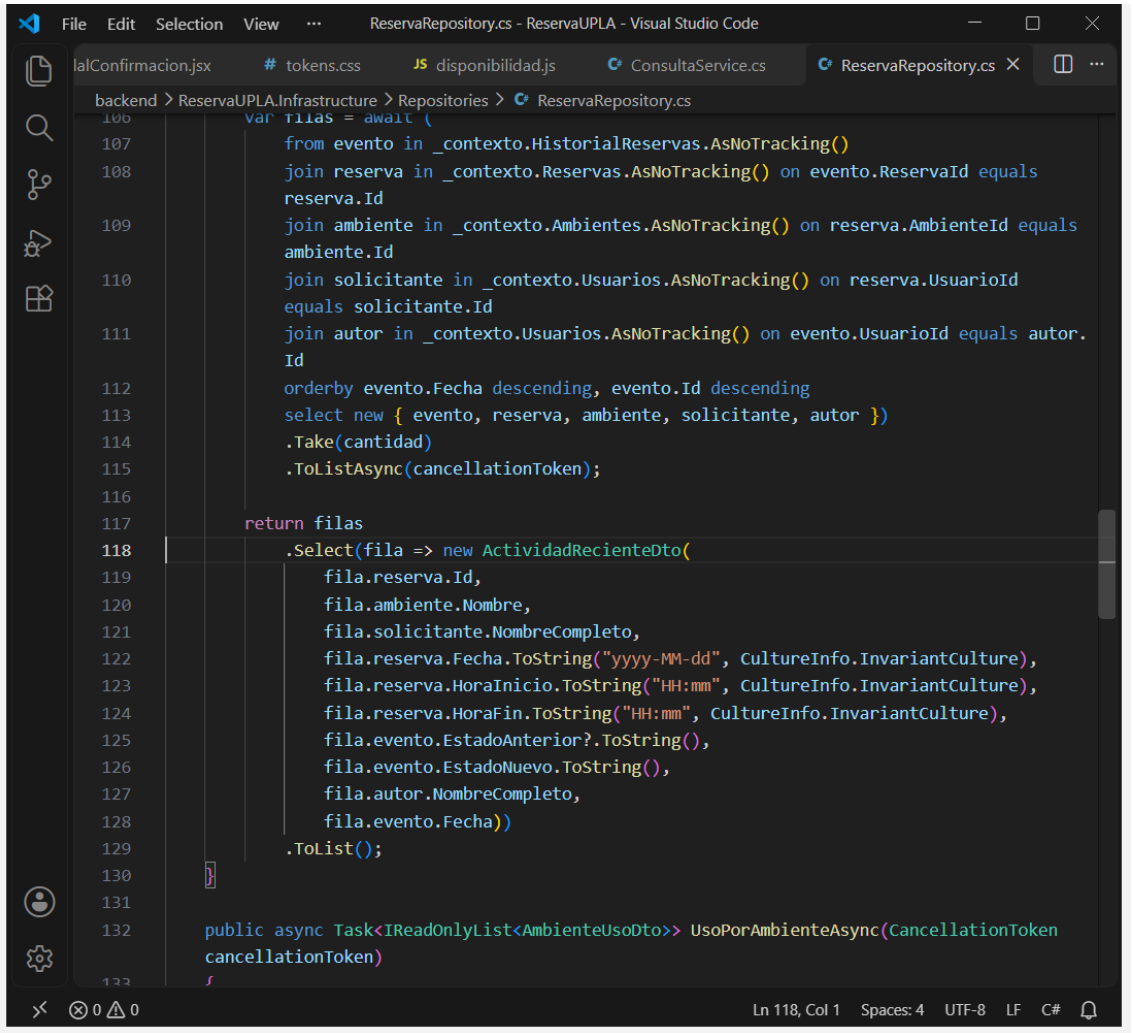
Figura 31. Confirmación antes de decidir: el modal dice lo que se va a hacer



9.3 Dashboard (RF-08)

Une el historial con la reserva, el ambiente y las dos personas implicadas y trae solo los últimos movimientos; el servicio de aplicación solo conoce la interfaz del repositorio.

Figura 32. Consulta de la actividad reciente del dashboard (Infrastructure) (backend/ReservaUPLA.Infrastructure/Repositories/ReservaRepository.cs, líneas 104–115)



El caso pide cinco indicadores como mínimo. Los cinco están, y se calculan en el servidor con consultas a la base de datos, no con datos de ejemplo.

Indicador del caso	Cómo se calcula	Dónde
Total de reservas	Cantidad de reservas registradas (incluye las canceladas, y la tarjeta lo dice)	backend/ ReservaUPLA.Infrastructure/Repositories/ ReservaRepository.cs:83
Reservas pendientes	Cantidad en estado Pendiente	backend/ ReservaUPLA.Infrastructure/Repositories/ ReservaRepository.cs:83

Indicador del caso	Cómo se calcula	Dónde
Reservas aprobadas	Cantidad en estado Aprobada	backend/ ReservaUPLA.Infrastructure/Repositories/ ReservaRepository.cs:83
Reservas rechazadas	Cantidad en estado Rechazada	backend/ ReservaUPLA.Infrastructure/Repositories/ ReservaRepository.cs:83
Ambiente más utilizado	El de más reservas aprobadas; si hay empate se muestran todos	backend/ ReservaUPLA.Infrastructure/Repositories/ ReservaRepository.cs:150

El servicio que arma todo el dashboard está en `backend/ReservaUPLA.Application/Services/ConsultaService.cs:59` y la pantalla del Administrador en `frontend/src/pages/InicioAdministrador.jsx:54`.

Figura 33. Inicio del Administrador: decisiones pendientes, agenda de hoy e indicadores

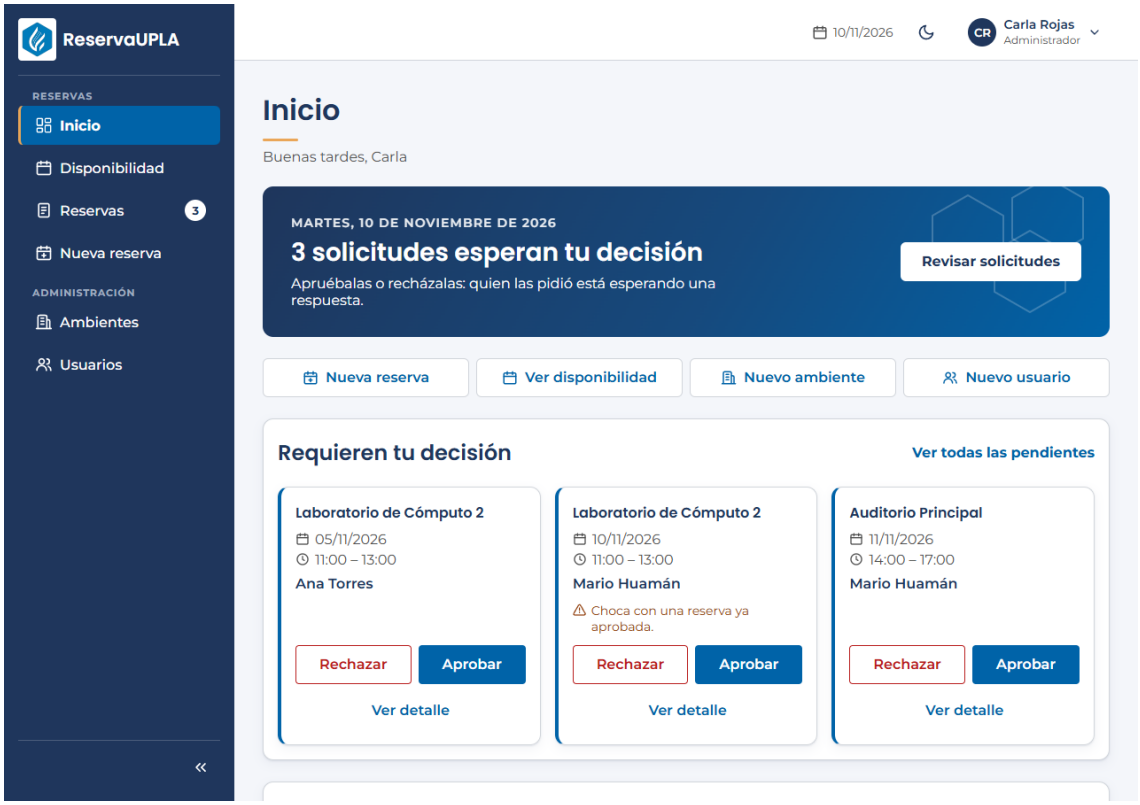
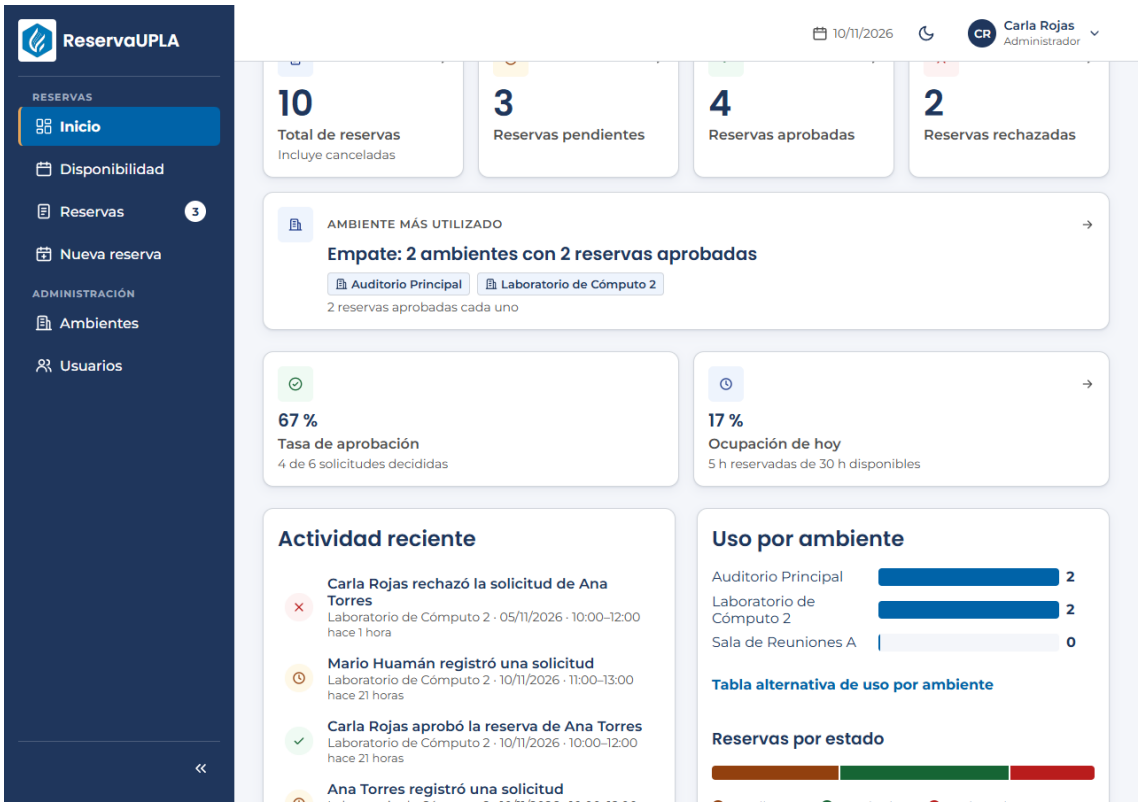


Figura 34. Los cuatro indicadores por estado, el ambiente más utilizado y los indicadores de gestión



9.4 Más allá del mínimo

- **Dos indicadores de gestión:** la tasa de aprobación (aprobadas ÷ aprobadas más rechazadas; las pendientes y canceladas no cuentan) y la ocupación de hoy (horas aprobadas ÷ horas disponibles de los ambientes que hoy no están en mantenimiento). Si no hay datos muestran «—» en vez de inventar un número. Cálculo en `frontend/src/utils/gestion.js`, con pruebas de borde.
- **Actividad reciente:** los últimos seis cambios de estado de cualquier reserva con quién los hizo y cuándo. Es la respuesta visible a la «falta de trazabilidad» del caso; el backend la expone solo al Administrador.
- **Accesos rápidos** a las cuatro tareas más frecuentes del Administrador.
- **Inicio del Solicitante y del Responsable**, cada uno con lo que necesita: su próxima reserva y el estado de sus solicitudes, o la agenda de su ambiente.
- **Vista diaria de disponibilidad** con tramos libres, aprobados, pendientes y en mantenimiento, que lleva a «Nueva reserva» con el ambiente, la fecha y la hora ya elegidos.

Figura 35. Actividad reciente: quién cambió el estado de qué reserva y cuándo

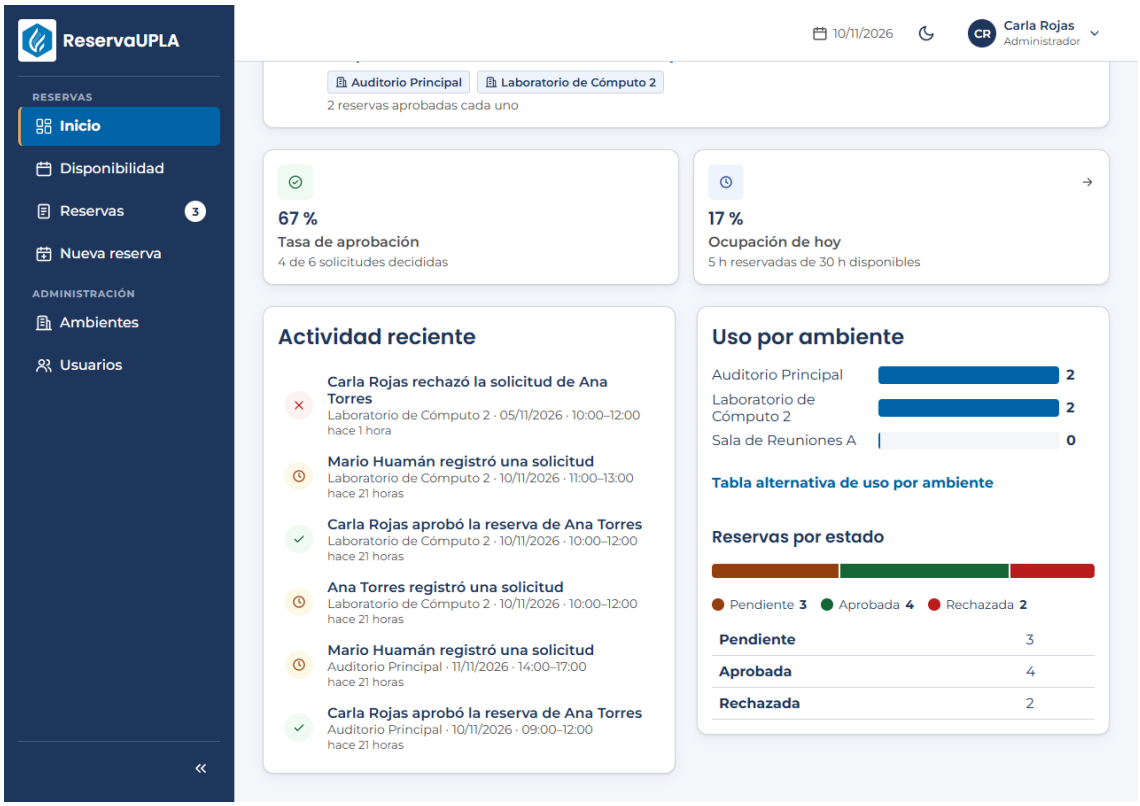


Figura 36. Inicio del Solicitante: su próxima reserva vigente y sus solicitudes por estado

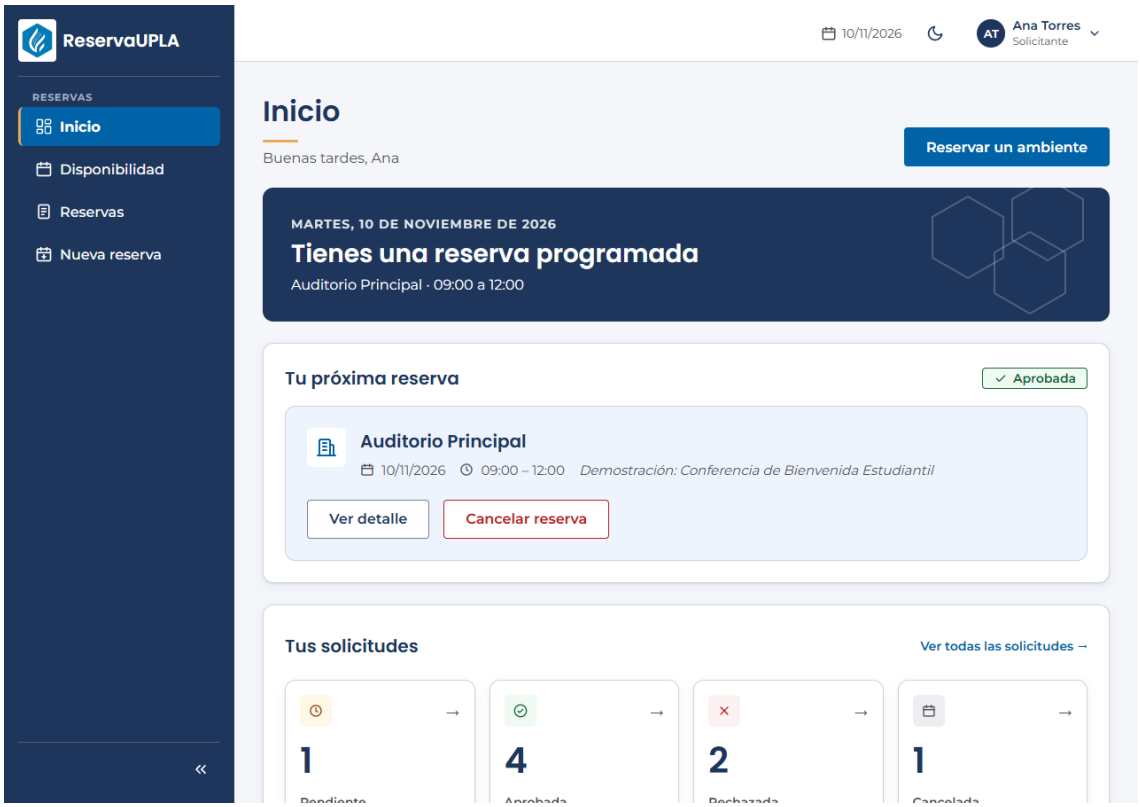


Figura 37. Inicio del Responsable de ambiente: solo consulta, sin acciones de decisión

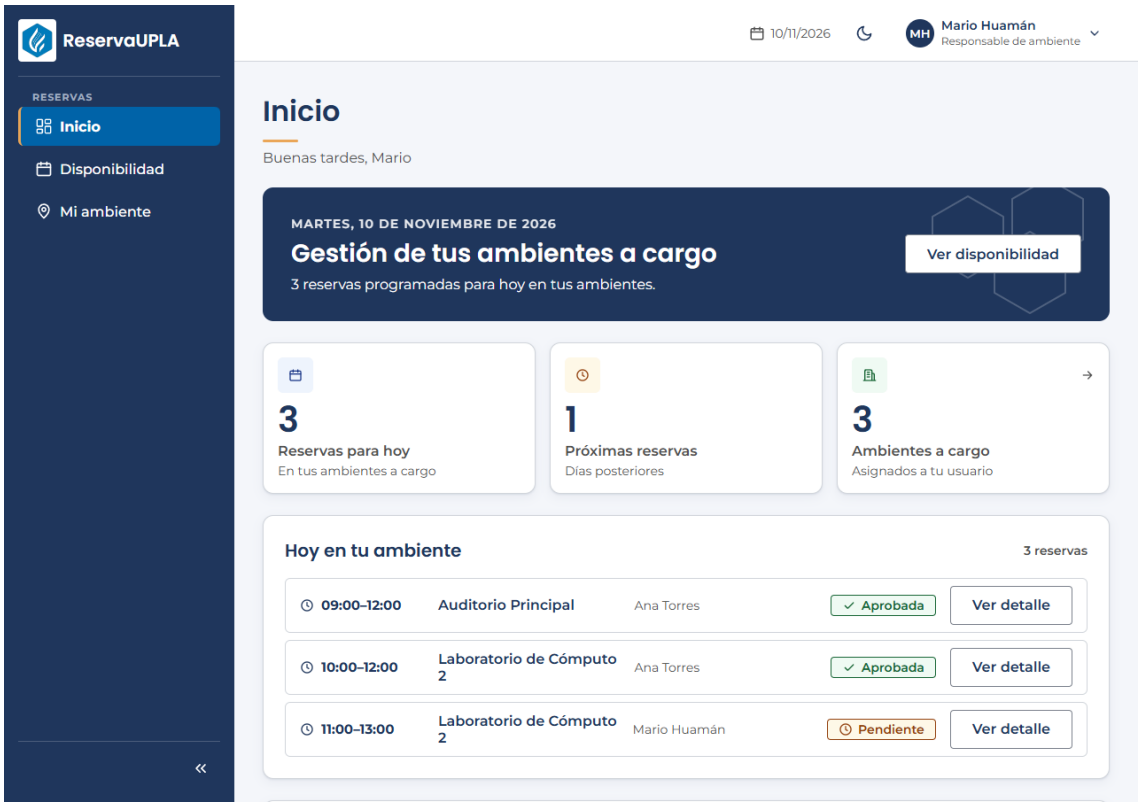


Figura 38. Vista diaria de disponibilidad por ambiente



10. Calidad: pruebas, accesibilidad, usabilidad y mantenibilidad

10.1 Pruebas automáticas

Cada cifra lleva su fecha, su comando y el entorno donde se obtuvo. Las que no pudieron repetirse tras los últimos cambios se dicen aparte.

Conjunto	Resultado	Fecha	Entorno y comando
Frontend (Vitest)	1375 aprobadas en 62 archivos, 0 fallos	2026-11-10	`npm --prefix frontend test`
Backend: aplicación (xUnit)	162 aprobadas	2026-11-10	WSL (Ubuntu), sin base de datos
Backend: infraestructura con base real	14 aprobadas	2026-11-10	WSL con PostgreSQL 18 real
Backend: API con base real	60 aprobadas	2026-11-07	Windows, con PostgreSQL real (60 de 60)
Herramientas de usabilidad (Node)	24 aprobadas	2026-11-10	`node --test pruebas/*.test.mjs (tools/usabilidad)`
Extremo a extremo	9 de 9 fases	2026-11-07 16:33	`npm run todo (tools/e2e)`
Compilación del frontend	JavaScript 130,02 kB y CSS 15,79 kB (gzip)	2026-11-10	`npm --prefix frontend run build (tamaños gzip)`

Limitación declarada. En la última etapa del proyecto, Windows activó Smart App Control en el equipo de desarrollo y bloqueó los programas compilados localmente (la API y las pruebas de .NET). Las pruebas de backend de 176 casos se ejecutaron entonces en WSL (Linux) con un PostgreSQL propio, incluidas las que usan base real. Las 60 pruebas del proyecto API y los guiones de extremo a extremo y de accesibilidad automatizada necesitan levantar la API en Windows, así que su **última ejecución completa es del 2026-11-07**, antes de agregar los indicadores de gestión, la actividad reciente, el login renovado y el modo oscuro. Esos cambios se verificaron con pruebas unitarias, el contraste medido y revisión en pantalla con datos reales, pero no con una nueva corrida completa.

10.2 Accesibilidad medida

Última medición completa de la interfaz con el navegador real (2026-11-07 16:23, `$env:ETAPA="taste"; node tools/usabilidad/accesibilidad.mjs`):

- axe-core: 0 violaciones en 87 estados (43 a 1280 px y 44 a 375 px).
- Contraste (WCAG 2.2 AA): 0 pares por debajo del umbral AA.
- 0 objetivos táctiles menores de 44 px (44 estados móviles).

- Teclado: 0 paradas de teclado sin foco visible.
- Zoom del 200 %: 0 de 30 combinaciones con desplazamiento horizontal a 188 px (zoom 200 %).
- Movimiento: 0 animaciones infinitas con prefers-reduced-motion.

Además, la herramienta de contraste del proyecto mide cada par de colores que declaran las hojas de estilo, en el tema claro y en el oscuro (0 pares por debajo del umbral en ambos).

10.3 Usabilidad: principios aplicados

Las decisiones de interfaz siguen la ISO 9241-11 (eficacia, eficiencia y satisfacción), las heurísticas de Nielsen y las reglas de oro de Shneiderman. Algunos ejemplos concretos, cada uno comprobado con una prueba:

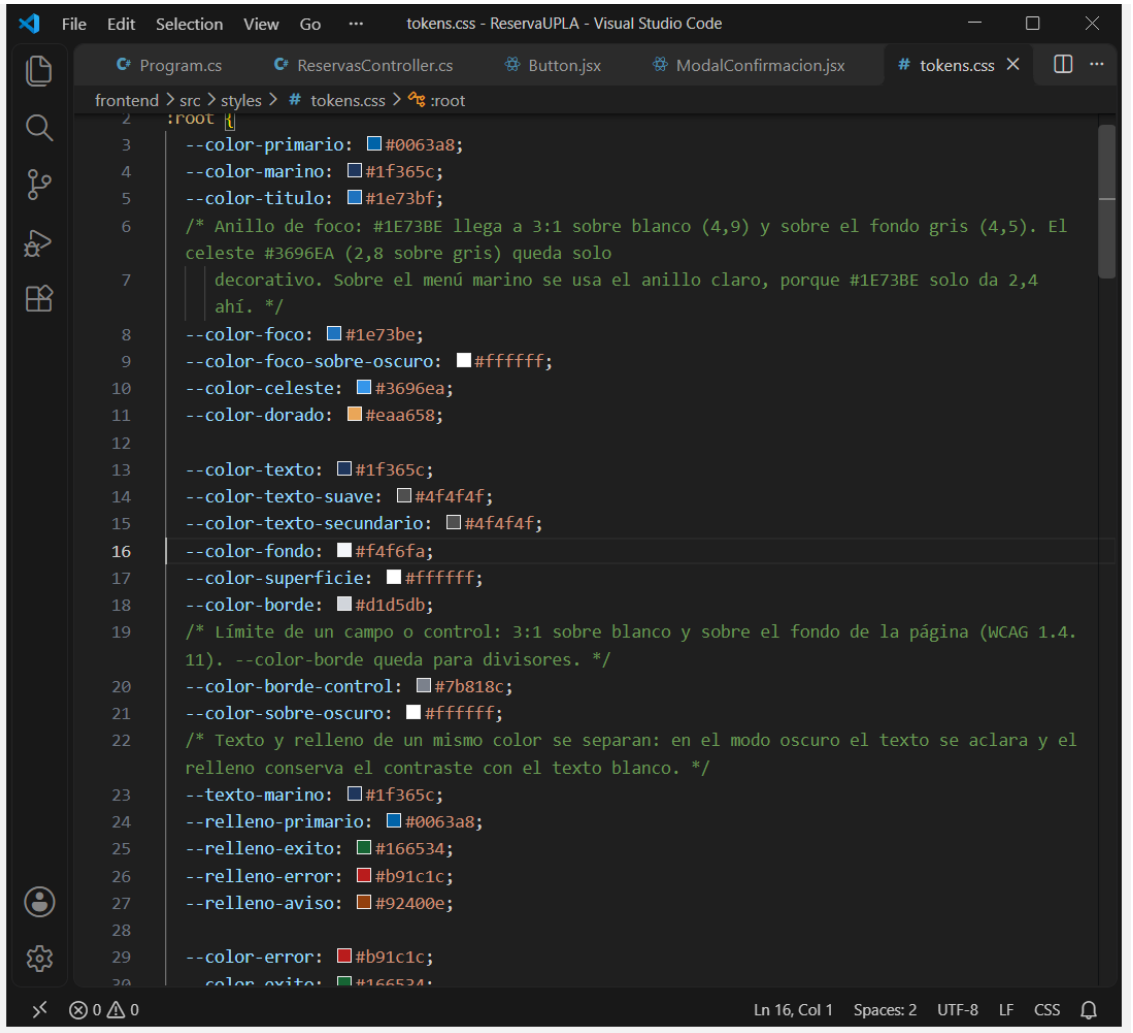
Principio	**Decisión en ReservaUPLA**
Visibilidad del estado (Nielsen 1)	Insignias con icono y texto; avisos tras cada acción («Reserva rechazada.»); esqueletos de carga con la forma de la tabla
Prevención de errores (Nielsen 5, Shneiderman 5)	Horas como lista cerrada, aviso de disponibilidad antes de enviar, botones deshabilitados con su razón visible
Control y libertad (Nielsen 3)	Confirmación antes de aprobar o rechazar, «Volver» en todo formulario y modo claro u oscuro a elección
Reconocimiento antes que recuerdo (Nielsen 6)	Etiquetas siempre visibles, ejemplos bajo los campos y el resumen en vivo de la solicitud
Diseño minimalista (Nielsen 8)	Cada pantalla tiene una acción principal; el Inicio dice una vez las solicitudes por decidir
Reducir la carga de memoria (Shneiderman 8)	Accesos rápidos, filtros recordados en la dirección de la página y atajos de duración

Pendiente de la estudiante: la prueba de usabilidad con tres a cinco compañeros (tareas, tiempos, errores y satisfacción) tiene su plantilla en `docs/usabilidad/plan-prueba-usuarios.md` y su hoja de resultados vacía en `docs/usabilidad/resultados-prueba.csv`. No se inventaron resultados: **[COMPLETAR POR EL ESTUDIANTE: resultados de la prueba con compañeros]**.

10.4 Modo claro y oscuro

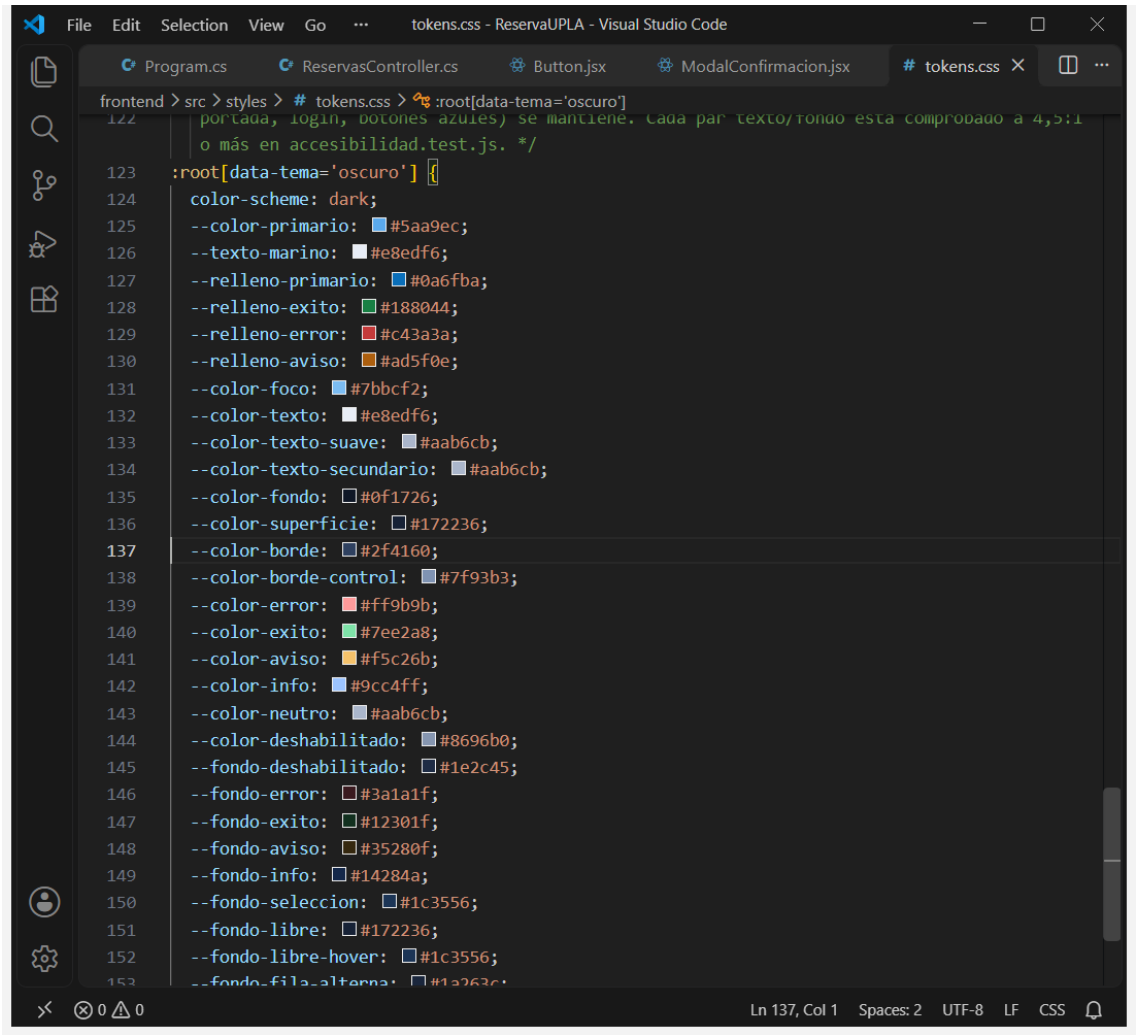
Ningún componente escribe un color suelto: todos usan una variable. Una prueba automática falla si alguien escribe un color fuera de este archivo.

Figura 39. Tokens de diseño: todos los colores salen de este archivo (frontend/src/styles/tokens.css, líneas 2–9)



El modo oscuro se agregó al final sin tocar un solo componente: basta con redefinir los tokens cuando <html> tiene data-tema="oscuro".

Figura 40. Modo oscuro: solo se redefinen las variables (frontend/src/styles/tokens.css, líneas 123–134)



Al final del proyecto se agregó un modo oscuro, y fue posible gracias a los estándares de la sección 3: como todos los colores viven en `tokens.css`, el tema oscuro es un bloque que redefine superficies, textos, bordes y estados sin tocar un solo componente. Se separó el color de texto del de relleno para poder aclarar solo los textos; cada par de colores del tema oscuro tiene una prueba de contraste (AA). El botón con luna o sol está en la barra superior y en el login, recuerda la elección y se aplica antes de pintar, sin parpadeo.

Figura 41. El mismo Inicio del Administrador en modo oscuro

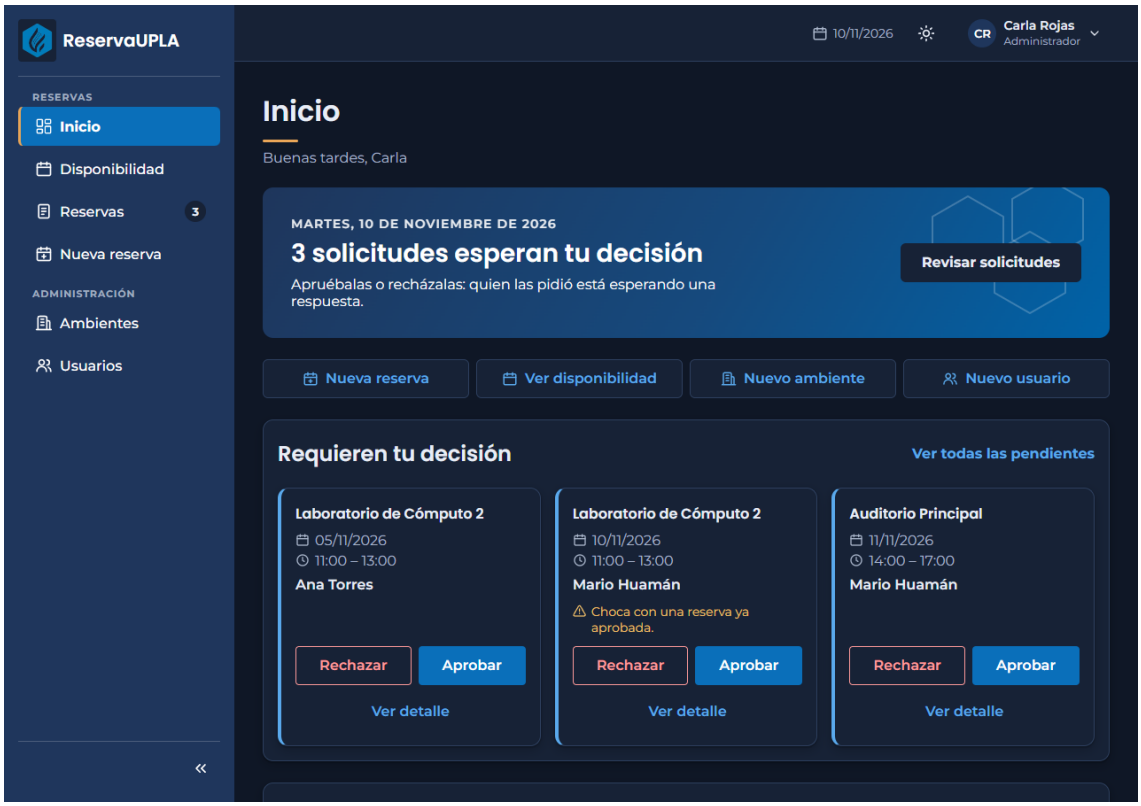
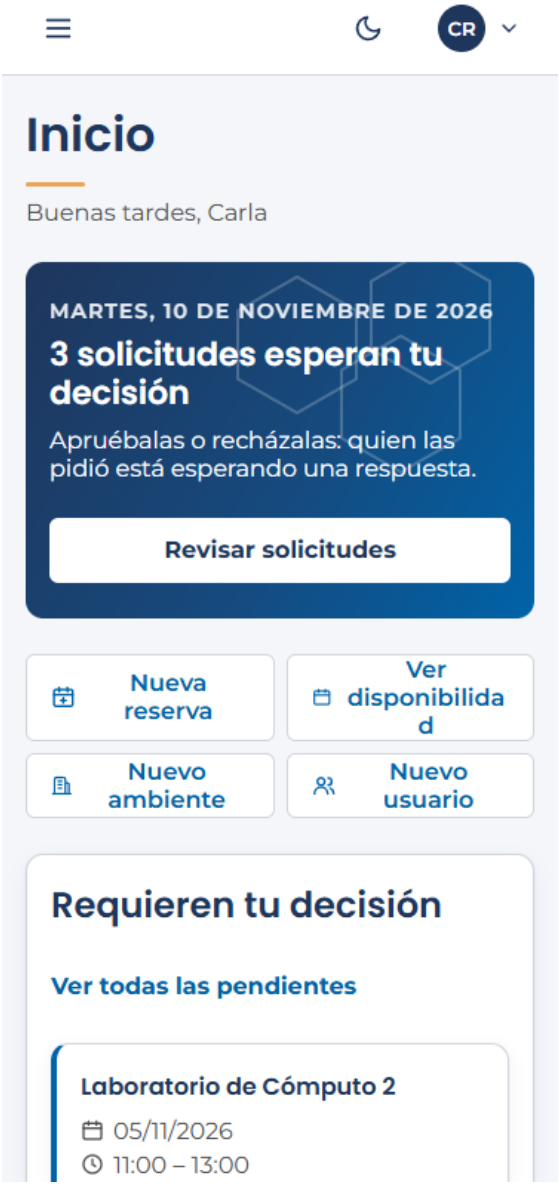


Figura 42. Pantalla de inicio de sesión (claro): foto del campus, tipos de ambiente y formulario



Figura 43. Inicio del Administrador en un teléfono (375 px): una columna, sin desplazamiento horizontal



10.5 Mantenibilidad y escalabilidad: qué se puede afirmar

Mantenibilidad (medida o comprobable): las capas del backend dependen en una sola dirección y una prueba lo verifica; los textos, colores y formas tienen una sola fuente cada uno; ningún archivo de código (JavaScript, JSX o C#) supera las 300 líneas; la duplicación de código es menor de 1 %; y hay una red de seguridad de más de 1 500 pruebas automáticas que avisa si un cambio rompe algo. La prueba más práctica es el cambio realizado al final: agregar un modo oscuro tocó los tokens y no los componentes.

Escalabilidad (con límites): hay pistas buenas: la autenticación JWT no guarda sesión en el servidor (permite varias instancias), la paginación y el alcance por rol se resuelven en la base, y los índices cubren las consultas por ambiente y fecha. Pero **no se midió con carga**: no hay una prueba con miles

de reservas ni con usuarios simultáneos. Por eso el informe afirma que el sistema está *preparado* para escalar, no que ya se haya demostrado a escala.

11. Reflexión final y conclusiones

11.1 La pregunta del caso

«¿Cómo contribuye la aplicación de estándares, la validación de datos, la reutilización de componentes y el refinamiento de código a mejorar la calidad, la mantenibilidad y la evolución del sistema ReservaUPLA?»

11.2 Respuesta

Antes de este proyecto yo pensaba que la calidad del software era sobre todo que «funcione». ReservaUPLA me enseñó que funcionar es solo el punto de partida: lo que se nota con el tiempo es si el sistema se puede **cambiar sin miedo**, y eso depende de las cuatro prácticas que pide el caso. Cada una aportó algo distinto y comprobable.

Los estándares dieron orden y evitaron discusiones inútiles. Tener decidido de antemano cómo se nombra, dónde va cada cosa, cómo se escribe un commit y qué capa puede depender de cuál hizo que el código fuera predecible: encontrar dónde se aplica una regla tardaba segundos. Y lo más valioso fue que muchos estándares se volvieron **pruebas automáticas**: una prueba falla si una pantalla escribe un botón a mano, si un color queda fuera de `tokens.css` o si la capa de aplicación referencia a la de infraestructura. Un estándar que nadie comprueba se olvida; uno que rompe la compilación se respeta.

La validación de datos en tres capas dio confianza. Al principio validar en el frontend me parecía suficiente, hasta que escribí la prueba de las dos aprobaciones simultáneas y vi que dos personas pueden pasar el mismo aviso en el mismo segundo. El frontend ayuda al usuario, el backend decide y la base de datos es la última barrera: solo la restricción de exclusión de PostgreSQL garantiza de verdad que nunca haya dos reservas aprobadas que se crucen. Aprendí a no confiar en lo que viene del cliente y a que un mensaje claro («Ese ambiente ya tiene una reserva aprobada de 10:00 a 12:00») es parte de la validación, no un adorno.

La reutilización redujo el costo de cada cambio. Con los nueve componentes y unos pocos hooks compartidos, un cambio se hace una vez y llega a todas las pantallas. La prueba más clara fue agregar el modo oscuro al final: como todos los colores salían de un solo archivo de tokens, no hubo que tocar ningún componente. Medí también la duplicación (de 1,38 % a 0,93 %) y entendí que el número importa menos que el hábito de preguntarse «¿esto ya existe?» antes de copiar.

El refinamiento me enseñó a desconfiar de mis propias pruebas. Refactorizar sin pruebas es un riesgo, y con ellas es una rutina. Pero lo que más aprendí fue otra cosa: con más de mil pruebas en verde, al levantar el sistema con datos reales aparecieron defectos que ninguna cubría, como que «Tu próxima reserva» mostrara una reserva cancelada o que un menú contraído quedara de 80 píxeles de alto. Refinar no es solo ordenar el código; es mirar el sistema funcionando, encontrar lo que falla y dejar una prueba para que no vuelva.

11.3 Qué mejoró en calidad, mantenibilidad y evolución

- **Calidad:** las reglas de negocio se cumplen aunque alguien se salte el formulario, y los mensajes son iguales en las tres capas.
- **Mantenibilidad:** capas con dependencias en una sola dirección, una sola fuente de textos, colores y formas, archivos cortos y una red de pruebas que avisa de inmediato si un cambio rompe algo.
- **Evolución:** se pudieron agregar, ya con el sistema terminado, una vista de disponibilidad, indicadores de gestión, un registro de actividad y un modo oscuro sin reescribir lo anterior. Cada una se apoyó en piezas que ya existían.

11.4 Limitaciones y trabajo pendiente

- No se midió la escalabilidad con carga (miles de reservas o usuarios simultáneos): el informe afirma que el sistema está preparado, no que se haya demostrado.
- La última ejecución completa de las pruebas de la API, del extremo a extremo y de la accesibilidad automatizada es del 7 de noviembre; los cambios posteriores se verificaron con pruebas unitarias y revisión en pantalla (sección 10.1).
- Falta la prueba de usabilidad con compañeros: **[COMPLETAR POR EL ESTUDIANTE: fecha, número de participantes y hallazgos]**.
- Fuera del alcance por decisión del análisis: notificaciones por correo, reservas recurrentes, calendario semanal o mensual, exportación y recuperación de contraseña.

Anexo A. Cómo ejecutar el sistema

A.1 Requisitos

- .NET 8, Node.js 20 o superior, PostgreSQL 16 o superior (con `psql`) y Git. En este equipo, PostgreSQL 18 escucha en `localhost:5432`.

A.2 Base de datos (por terminal, desde la raíz del repositorio)

En PowerShell. La clave de `postgres` se define solo para la sesión; la del rol de la aplicación debe tener 24 caracteres o más y guardarse solo en `user-secrets`.

```
$env:PGPASSWORD = '<clave de postgres>'
```

```
$psql = @('-h','localhost','-p','5432','-U','postgres','-v','ON_ERROR_STOP=1')
```

```
psql @psql -d postgres -f database/00_eliminar_bd.sql
```

```
psql @psql -d postgres -f database/01_crear_bd.sql
```

```
psql @psql -d postgres -v app_password="<clave del rol>" -f
database/02_crear_rol.sql
```

```
psql @psql -d ReservaUPLADB -f database/03_crear_tablas.sql
```

```
psql @psql -d ReservaUPLADB -f database/04_permisos.sql
```

```
psql @psql -d ReservaUPLADB -v password_prueba="<clave de prueba>" -f
database/05_datos_prueba.sql
```

```
psql @psql -d ReservaUPLADB -f database/06_verificacion.sql
```

`psql @psql -d ReservaUPLADB -f database/07_datos_demo.sql` (opcional: reservas con fechas de hoy para la demostración)

A.3 Secretos

Desde `backend/`:

```
dotnet user-secrets set "ConnectionStrings:ReservaUPLADB"
"Host=localhost;Port=5432;Database=ReservaUPLADB;Username=reservaupla_app;Password
=<clave del rol>" --project ReservaUPLA.API/ReservaUPLA.API.csproj
```

```
dotnet user-secrets set "Jwt:Key" "<cadena aleatoria de 64 caracteres o más>" --
project ReservaUPLA.API/ReservaUPLA.API.csproj
```

A.4 Backend y frontend

Terminal 1 (API en `http://localhost:5263`): `cd backend` y luego `$env:ASPNETCORE_ENVIRONMENT='Development'; dotnet run --project ReservaUPLA.API/ReservaUPLA.API.csproj --urls http://localhost:5263`.

Terminal 2 (frontend en `http://localhost:5180`): `cd frontend`, `npm install` y `npm run dev`.

A.5 Cuentas de prueba

La contraseña de las tres cuentas está en la sección «Datos de prueba» del README del repositorio (no se repite aquí: son datos de prueba solo para el entorno de desarrollo). El inicio de sesión permite cinco intentos por minuto.

Rol	Correo	Nombre
Administrador	`carla.rojas@upla.edu.pe`	Carla Rojas
Solicitante	`ana.torres@upla.edu.pe`	Ana Torres
Responsable de ambiente	`mario.huaman@upla.edu.pe`	Mario Huamán

A.6 Pruebas

Frontend: `cd frontend; npm run lint; npm test; npm run build`. Backend: `cd backend; dotnet test ReservaUPLA.sln` (las pruebas con base real necesitan las variables `RESERVAUPLA_PRUEBAS_BD_ADMIN` y `RESERVAUPLA_PRUEBAS_BD_APP`). Base de datos: `psql @psql -d ReservaUPLADB -f database/07_pruebas_reglas.sql`. Extremo a extremo y accesibilidad: `cd tools/e2e; npm run todo` y `cd tools/usabilidad; node accesibilidad.mjs`.

A.7 Documentos de evidencia en el repositorio

Documento	Contenido
`docs/01-analisis.md`, `02-ciclo-de-vida.md`, `03-estandares.md`	Semanas 01 a 03
`docs/matriz-reglas.md`	Cada regla, en cada capa, con el nombre exacto de la prueba que la rompe a propósito
`docs/reutilizacion.md`	Duplicación antes y después y componentes por pantalla
`docs/bitacora-evaluacion.md`	Cada defecto hallado, su corrección y la prueba que lo cubre
`docs/diseno/` y `docs/usabilidad/`	Identidad visual, experiencia, componentes, matriz de principios y plan de prueba de usabilidad
`docs/evidencias/`	Capturas, informes de accesibilidad y de peso

Anexo B. Índice de figuras

Las capturas son del sistema real en marcha (con la base cargada con `database/07_datos_demo.sql`) y las genera `tools/generar-informe/capturas.mjs`; los archivos están en `docs/evidencias/informe/`.

Figura	Título	Sección	Archivo
1	Prueba de arquitectura: Application no puede referenciar Infrastructure (backend/ ReservaUPLA.Application.Tests/ArquitecturaTests.cs, líneas 12–21)		`codigo-vscode/ arquitectura-test.png`
2	Límite de intentos del inicio de sesión: 5 por minuto por dirección IP (backend/ReservaUPLA.API/ Program.cs, líneas 79–92)		`codigo-vscode/ ratelimit.png`
3	Registro de validadores y servicios de la capa Application (backend/ReservaUPLA.Application/ DependencyInjection.cs, líneas 18–31)		`codigo-vscode/inyeccion-application.png`
4	Permisos del rol de la aplicación (mínimo privilegio) (database/04_permisos.sql, líneas 12–17)		`codigo-vscode/sql-permisos.png`
5	Tabla reservas: restricciones NOT NULL, CHECK y de exclusión (database/03_crear_tablas.sql, líneas 41–61)		`codigo-vscode/sql-tabla-reservas.png`
6	Script que prueba las reglas directamente en PostgreSQL (database/07_pruebas_reglas.sql, líneas 51–56)		`codigo-vscode/sql-pruebas-reglas.png`
7	Cliente HTTP del frontend: tiempo límite, cancelación y errores de red (frontend/src/services/apiClient.js, líneas 43–70)		`codigo-vscode/ apiclient.png`
8	Hook useFormulario: el estado de los formularios está separado de su presentación (frontend/src/hooks/ useFormulario.js, líneas 22–45)		`codigo-vscode/hook-formulario.png`
9	Formulario «Nueva reserva» recién abierto (Solicitante)	5	`20-nueva-reserva.png`
10	Formulario con un ambiente elegido, horario y resumen en vivo	5	`22-nueva-reserva-completa.png`
11	Formulario «Registrar ambiente» con un error de validación (Administrador)	5	`24-ambiente-nuevo-errores.png`

Figura	Título	Sección	Archivo
12	Formulario «Registrar usuario» con tres errores a la vez (Administrador)	5	25-usuario-nuevo-errores.png
13	CampoFormulario: etiqueta, caja, ayuda, error y aviso de todos los campos (frontend/src/components/ui/CampoFormulario.jsx, líneas 38–65)		codigo-vscode/campo-formulario.png
14	Validación del correo en el frontend (función pura) (frontend/src/utils/validators.js, líneas 37–48)		codigo-vscode/validador-correo.png
15	Validador de la reserva en el backend (FluentValidation) (backend/ReservaUPLA.Application/Validators/CrearReservaValidator.cs, líneas 15–26)		codigo-vscode/validador-reserva.png
16	Mensajes de validación en una sola clase (backend/ReservaUPLA.Application/Common/MensajesValidacion.cs, líneas 14–19)		codigo-vscode/mensajes.png
17	AuthService: el mismo resultado si el correo no existe, está inactivo o la clave es incorrecta (backend/ReservaUPLA.Application/Services/AuthService.cs, líneas 38–61)		codigo-vscode/auth-servicio.png
18	ReservaService: comprobación de cruce antes de aprobar (RN-01) (backend/ReservaUPLA.Application/Services/ReservaService.cs, líneas 200–209)		codigo-vscode/servicio-cruce.png
19	Entidad Reserva: las transiciones de estado viven en el dominio (RN-06) (backend/ReservaUPLA.Domain/Entities/Reserva.cs, líneas 40–58)		codigo-vscode/entidad-reserva.png
20	Enviar «Nueva reserva» vacía: un mensaje junto a cada campo y el foco en el primero	6	21-nueva-reserva-errores.png
21	RN-01: aviso de disponibilidad con un tramo ocupado marcado «Choque»	6	23-nueva-reserva-ocupado.png
22	Credenciales incorrectas: mensaje genérico que no revela si el correo existe	6	04-login-error.png
23	Button: bloqueo del doble clic mientras se procesa (frontend/src/components/ui/Button.jsx, líneas 21–28)		codigo-vscode/boton.png
24	Corrección de un defecto real: el texto de cada tramo depende de su ancho (frontend/src/utils/disponibilidad.js, líneas 137–150)		codigo-vscode/texto-bloque.png

Figura	Título	Sección	Archivo
25	Corrección de un defecto real: «Tu próxima reserva» ya no muestra una cancelada (backend/ ReservaUPLA.Application/Services/ConsultaService.cs, líneas 76–80)		codigo-vscode/proxima-reserva.png
26	Listado de reservas con pestañas por estado, filtros y tabla (Administrador)	9	12-reservas.png
27	Controlador delgado: autoriza, delega en el servicio y traduce el resultado a HTTP (backend/ ReservaUPLA.API/Controllers/ReservasController.cs, líneas 91–96)		codigo-vscode/ controlador.png
28	ModalConfirmacion: props del cuadro de confirmación reutilizable (frontend/src/components/ ui/ModalConfirmacion.jsx, líneas 13–25)		codigo-vscode/modal.png
29	Detalle de una reserva pendiente con su historial y acciones (Administrador)	9	15-detalle-pendiente.png
30	Reserva pendiente que choca con una aprobada: «Aprobar» queda deshabilitado y se explica por qué	9	16-detalle-con-choque.png
31	Confirmación antes de decidir: el modal dice lo que se va a hacer	9	17-modal-aprobar.png
32	Consulta de la actividad reciente del dashboard (Infrastructure) (backend/ ReservaUPLA.Infrastructure/Repositories/ ReservaRepository.cs, líneas 104–115)		codigo-vscode/actividad-reciente.png
33	Inicio del Administrador: decisiones pendientes, agenda de hoy e indicadores	9	05-inicio-admin.png
34	Los cuatro indicadores por estado, el ambiente más utilizado y los indicadores de gestión	9	06-inicio-admin-indicadores.png
35	Actividad reciente: quién cambió el estado de qué reserva y cuándo	9	07-inicio-admin-actividad.png
36	Inicio del Solicitante: su próxima reserva vigente y sus solicitudes por estado	9	09-inicio-solicitante.png
37	Inicio del Responsable de ambiente: solo consulta, sin acciones de decisión	9	10-inicio-responsable.png
38	Vista diaria de disponibilidad por ambiente	9	14-disponibilidad.png

Figura	Título	Sección	Archivo
39	Tokens de diseño: todos los colores salen de este archivo (frontend/src/styles/tokens.css, líneas 2–9)		codigo-vscode/tokens-claro.png
40	Modo oscuro: solo se redefinen las variables (frontend/src/styles/tokens.css, líneas 123–134)		codigo-vscode/tokens-oscuro.png
41	El mismo Inicio del Administrador en modo oscuro	10	08-inicio-admin-oscuro.png
42	Pantalla de inicio de sesión (claro): foto del campus, tipos de ambiente y formulario	10	01-login.png
43	Inicio del Administrador en un teléfono (375 px): una columna, sin desplazamiento horizontal	10	11-inicio-admin-movil.png